DitaCraft User Guide

The Complete Guide to DITA Editing in VS Code

DitaCraft User Guide

The Complete Guide to DITA Editing in VS Code

DitaCraft User Guide

The Complete Guide to DITA Editing in VS Code

Version 0.6.2

March 2026

Jeremy Jeanne

DitaCraft Project

Contents

Preface

Preface

Welcome to the DitaCraft User Guide.

About This Guide

This guide provides comprehensive documentation for DitaCraft, the Visual Studio Code extension for DITA authoring and publishing. Whether you are new to DITA or an experienced technical writer, this guide will help you get the most out of DitaCraft.

Intended Audience

This guide is intended for:

- **Technical Writers** who create and maintain DITA documentation
- **Documentation Managers** who oversee DITA projects
- **Developers** who write technical documentation alongside code
- **Content Strategists** evaluating DITA tooling options

Prerequisites

To use DitaCraft effectively, you should have:

- Visual Studio Code 1.80 or higher installed
- Basic familiarity with XML concepts
- Understanding of DITA fundamentals (helpful but not required)

How to Use This Guide

This guide is organized into the following sections:

Part I: Getting Started	Introduction to DitaCraft and installation steps
Part II: Using DitaCraft	Commands reference and detailed feature explanations
Part III: Configuration	Settings and customization options
Appendix: Keyboard Shortcuts	Quick reference for all keyboard shortcuts
Glossary	Definitions of key terms used throughout the guide

Conventions Used

This guide uses the following conventions:

Convention	Meaning
Bold text	UI elements, buttons, menu items
`Monospace`	Code, commands, file names, settings
`File paths`	Directory and file paths
`User input`	Text you should type

Version Information

This guide documents DitaCraft version 0.9.0. For the latest updates and release notes, visit the project repository.

Notice

Notices

Legal notices and trademark information.

Copyright

Trademarks

The following are trademarks or registered trademarks of their respective owners:

- **Visual Studio Code** is a trademark of Microsoft Corporation
- **DITA** and **Darwin Information Typing Architecture** are trademarks of OASIS
- **DITA Open Toolkit** is a trademark of the DITA-OT Project

All other trademarks are the property of their respective owners.

Disclaimer

This documentation is provided "as is" without warranty of any kind, express or implied. The authors make no representations about the suitability of this information for any purpose.

Part
I

Getting Started

Topics:

Chapter 1

Introduction to DitaCraft

DitaCraft is a comprehensive Visual Studio Code extension for editing and publishing DITA content.

What is DitaCraft?

DitaCraft is the easiest way to edit and publish your DITA (Darwin Information Typing Architecture) files directly from Visual Studio Code. It provides a complete authoring environment with:

- LSP-powered IntelliSense with auto-completion, hover docs, and document symbols
- Full DTD validation with three engine options
- Smart navigation with Ctrl+Click on href, conref, keyref, and conkeyref
- Cross-file Go to Definition, Find References, and Rename
- Full DITA 1.3 spec-compliant key space construction: keyref chains, keyscope PushDown inheritance, inline scope branches, provenance tracking, and scope explosion protection
- 43 DITA validation rules with DITA 2.0 awareness
- Localized diagnostics (English, French)
- Explicit root map selection with status bar indicator
- DITAVAL conditional processing support
- Live HTML5 preview with bidirectional scroll sync
- One-click publishing via DITA-OT
- Interactive map visualizer
- XML formatting with code actions and quick fixes

Key Features

DitaCraft offers the following key features:

LSP-Powered IntelliSense	Context-aware auto-completion for DITA elements, attributes, and values. Hover documentation for 364+ elements. Document outline and workspace symbol search.
Smart Navigation	Ctrl+Click on `href`, `conref`, `keyref`, and `conkeyref` attributes to navigate directly to referenced content. Cross-file Go to Definition, Find References, and Rename.
Full DTD Validation	Real-time validation with three engine options: built-in (default),

	TypesXML (full DTD), and xmllint. LSP-powered diagnostics for instant feedback as you type.
Live Preview	Side-by-side HTML5 preview with bidirectional scroll sync, theme support, and print mode.
DITA-OT Integration	Publish to HTML5, PDF, EPUB, and more with direct DITA Open Toolkit integration.
DITAVAL Support	Full IntelliSense for DITAVAL conditional processing files including element completion, attribute values, and action keywords.
Map Visualizer	Interactive tree view showing your DITA map hierarchy with navigation support.

Supported File Types

DitaCraft supports the following DITA file types:

Extension	Description
`.dita`	DITA topics (concept, task, reference, topic)
`.ditamap`	DITA maps for organizing topics
`.bookmap`	Book-oriented maps with chapters and parts
`.ditaval`	DITAVAL conditional processing profiles

Version Information

This guide covers DitaCraft version 0.6.2, which includes:

- LSP server with IntelliSense, diagnostics, and navigation
- Cross-reference validation for href, conref, keyref, and conkeyref
- DITA Rules Engine with 43 Schematron-equivalent rules (including 10 DITA 2.0-specific rules)
- Profiling validation with subject scheme support and hierarchy grouping
- Activity Bar views: DITA Explorer, Key Space, and Diagnostics
- Explicit root map selection with status bar indicator
- Localized diagnostics (English and French)
- RNG (RelaxNG) and OASIS Catalog validation services
- Conref/conkeyref content preview on hover
- 9 code actions including accessibility quick fixes
- DITAVAL conditional processing support
- DITA version detection and error-tolerant XML tokenizer
- Smart debouncing (300ms topics, 1000ms maps) with per-document cancellation
- Key scope support with `@keyscope` attribute handling

- 1040+ comprehensive tests (620 client + 419 server)

Chapter 2

Getting Started

Learn how to install and configure DitaCraft for your first DITA project.

Before you begin

Before installing DitaCraft, ensure you have:

- Visual Studio Code 1.80 or higher
- DITA-OT 4.2.1 or higher (for publishing features)

About this task

This guide walks you through installing DitaCraft and creating your first DITA file.

Procedure

1. Install DitaCraft from VS Code Marketplace
   a) Open VS Code
   b) Press **Ctrl+P** (or **Cmd+P** on Mac)
   c) Type `ext install ditacraft` and press Enter
2. Configure DITA-OT path (optional, required for publishing)
   a) Download DITA-OT from dita-ot.org
   b) Extract to a location (e.g., `C:\DITA-OT-4.2.1`)
   c) Open Command Palette (**Ctrl+Shift+P**)
   d) Type `DITA: Configure DITA-OT Path`
   e) Select your DITA-OT installation directory
3. Create your first DITA topic
   a) Open Command Palette (**Ctrl+Shift+P**)
   b) Type `DITA: Create New Topic`
   c) Select topic type (concept, task, or reference)
   d) Enter a file name for your topic

   A new DITA topic file is created with proper DOCTYPE and structure.
4. Validate your DITA file
   a) Open your DITA file
   b) Press **Ctrl+Shift+V** to validate

   Validation results appear in the Problems panel. Errors and warnings are highlighted in the editor.
5. Preview your content (optional)
   a) Press **Ctrl+Shift+H** to open HTML5 preview

   A side-by-side preview panel opens showing your rendered DITA content.

Results

You now have DitaCraft installed and configured. You can:

- Create DITA topics, maps, and bookmaps
- Validate your content in real-time
- Navigate between files using Ctrl+Click
- Preview and publish your documentation

Part
II

Using DitaCraft

Topics:

Chapter 3

Commands Overview

DitaCraft provides commands for validation, publishing, file creation, and navigation.

Accessing Commands

All DitaCraft commands are accessible via the Command Palette:

1. Press **Ctrl+Shift+P** (or **Cmd+Shift+P** on Mac)
2. Type `DITA:` to see all available commands
3. Select the desired command

Command Categories

DitaCraft commands are organized into the following categories:

Validation Commands	Validate DITA files for XML syntax and DTD conformance
Publishing Commands	Publish DITA content to HTML5, PDF, and other formats
File Creation Commands	Create new DITA topics, maps, and bookmaps — or scaffold a whole new project at once
Editing and Refactoring Commands	Rename, move, extract, and inline content; insert images and tables; apply multi-file edits
Navigation Commands	Navigate and visualize DITA content structure

Quick Command Reference

Command	Shortcut	Description
DITA: Validate Current File	Ctrl+Shift+V	Validate the active DITA file
DITA: Publish (Select Format)	Ctrl+Shift+B	Publish with format selection
DITA: Publish to HTML5	-	Quick publish to HTML5
DITA: Preview HTML5	Ctrl+Shift+H	Show live HTML5 preview

Command	Shortcut	Description
DITA: Show Map Visualizer	-	Display interactive map hierarchy
DITA: Create New Topic	-	Create a new DITA topic
DITA: Create New Map	-	Create a new DITA map
DITA: Create New Bookmap	-	Create a new bookmap
DITA: Configure DITA-OT Path	-	Set DITA-OT installation path
DITA: Set Root Map	-	Set an explicit root map for key resolution
DITA: Clear Root Map (Auto-Discover)	-	Return to automatic root map discovery
DITA: Open File	-	Open a file from the DITA Explorer tree
DITA: Initialize DITA Project	-	Scaffold a new project: root map, starter topics, folder layout
Rename Symbol	F2	Rename an ID or key and update every reference workspace-wide
DITA: Extract Topic from Section	-	Extract a section into a new topic, replaced with an xref
DITA: Inline Conref	-	Splice a conref/ conkeyref target's content in place
DITA: Insert Image	-	Insert an image element, optionally wrapped in a fig
DITA: Insert Table	-	Insert a simpletable or CALS table skeleton
DITA: Find and Replace in Files	-	Multi-file search and replace with Refactor Preview
DITA: Batch Update Metadata	-	Set or clear a profiling attribute across selected files
DITA: Edit DITAVAL Conditions	-	Open the visual DITAVAL condition editor

Command	Shortcut	Description
DITA: Manage Publishing Profiles	-	Create, edit, or select a saved publish configuration
DITA: Start Watch Mode	-	Automatically republish whenever a watched file changes

Chapter 4

Validation Commands

Commands for validating DITA files against XML syntax and DTD specifications.

DITA: Validate Current File

Shortcut: Ctrl+Shift+V (Windows/Linux) or **Cmd+Shift+V** (Mac)

Validates the currently active DITA file for:

- XML well-formedness (proper tags, attributes, encoding)
- DTD conformance (valid elements, required attributes)
- DITA structure (required elements like title, proper nesting)
- Content model validation (correct child elements)

Usage:

1. Open a `.dita`, `.ditamap`, or `.bookmap` file
2. Press **Ctrl+Shift+V** or run the command from Command Palette
3. View results in the Problems panel

Output:

- **Errors** - Critical issues that must be fixed (red squiggles)
- **Warnings** - Potential issues to review (yellow squiggles)
- **Information** - Suggestions and notes (blue squiggles)

Automatic Validation

DitaCraft automatically validates DITA files when:

- **On Open:** When you open a DITA file
- **On Save:** When you save a DITA file (if `autoValidate` is enabled)
- **On Change:** After you stop typing, with smart debouncing (300ms for topics, 1000ms for maps)

To disable automatic validation, set `ditacraft.autoValidate` to `false` in settings.

Validation Engines

DitaCraft supports three validation engines:

Engine	Description	Requirements
`typesxml`	Full DTD validation with 100% W3C conformance (default, recommended)	None (bundled)

Engine	Description	Requirements
`built-in`	Lightweight XML and content model checking	None (bundled)
`xmllint`	External validation using libxml2	xmllint installed on system

To change the validation engine, set `ditacraft.validationEngine` in settings.

Rate Limiting

To prevent performance issues from rapid validation requests, DitaCraft includes rate limiting:

- Maximum 10 validation requests per second per file
- Excess requests are silently skipped (auto-validation) or show a warning (manual validation)
- Rate limits reset after the time window expires

DITA: Validate Entire Guide Using DITA-OT

Runs the DITA Open Toolkit against the configured root map to perform full end-to-end guide validation. This command detects cross-file issues that file-level validation cannot catch.

Prerequisites:

- DITA-OT must be installed and configured (`ditacraft.ditaOtPath`)
- A root map must be set (`ditacraft.rootMap` or via **DITA: Set Root Map**)

Output:

- **WebView Report** — Interactive panel with severity filtering, search, grouping (by file, severity, or module), error code tooltips, file navigation, and JSON export
- **Problems Panel** — Errors and warnings appear with `DITA-OT` source label and error codes

See Guide Validation (DITA-OT) on page 89 for detailed documentation.

Chapter 5

Publishing Commands

Commands for publishing DITA content to HTML5, PDF, and other output formats.

DITA: Publish (Select Format)

Shortcut: Ctrl+Shift+B (Windows/Linux) or **Cmd+Shift+B** (Mac)

Opens a format selection dialog and publishes the current DITA map or topic.

Available Formats:

- **html5** - Modern HTML5 output with responsive design
- **pdf** - PDF output (requires Apache FOP or similar)
- **xhtml** - XHTML 1.0 output
- **eclipsehelp** - Eclipse help system format
- **htmlhelp** - Microsoft HTML Help format
- **markdown** - Markdown output

Requirements:

- DITA-OT must be installed and configured
- Java Runtime Environment (JRE) 11 or higher

DITA: Publish to HTML5

Quickly publishes the current file to HTML5 format without format selection.

Usage:

1. Open a `.ditamap` or `.bookmap` file
2. Run the command from Command Palette
3. Output is generated in the `out` folder

Output Location:

By default, output is generated in `{workspace}/out/{format}`. You can customize this in settings with `ditacraft.outputDirectory`.

DITA: Preview HTML5

Shortcut: Ctrl+Shift+H (Windows/Linux) or **Cmd+Shift+H** (Mac)

Opens a live HTML5 preview panel alongside the editor.

Features:

- **Live Updates:** Preview refreshes automatically as you type
- **Scroll Sync:** Bidirectional scroll synchronization between editor and preview
- **Theme Support:** Light and dark theme support

- **Print Mode:** Toggle print-friendly styling
- **Conref Resolution:** Displays resolved conref content

Tip: The preview uses a lightweight renderer that does not require DITA-OT, making it instant for quick content review.

DITA: Manage Publishing Profiles

Create, edit, delete, or select a saved publish configuration — transtype, output directory, DITAVAL filter, and extra DITA-OT arguments — instead of re-entering the same options on every publish. See Publishing Workflow Enhancements on page 107 for the full field reference.

DITA: Start Watch Mode / DITA: Stop Watch Mode

Starts (or stops) automatically re-running a full publish whenever a watched DITA file changes, using the last-used publishing profile when one exists. See Publishing Workflow Enhancements on page 107 for target-resolution order and status bar behavior.

DITA: Configure DITA-OT Path

Opens a folder picker to select your DITA-OT installation directory.

Steps:

1. Download DITA-OT from dita-ot.org
2. Extract to a location on your system
3. Run this command and select the DITA-OT directory

Verification:

After configuration, DitaCraft verifies the installation by checking for:

- `bin/dita` or `bin/dita.bat` executable
- `lib/` directory with required JAR files
- Valid version information

Common Publishing Errors

If publishing fails, check the following:

DITA-OT not found	Run **DITA: Configure DITA-OT Path** to set the correct path.
Java not found	Install JRE 11+ and ensure `JAVA_HOME` is set correctly.
Build failed	Check the Output panel for detailed error messages from DITA-OT.

Chapter 6

File Creation Commands

Commands for creating new DITA topics, maps, and bookmaps.

DITA: Create New Topic

Creates a new DITA topic file with proper DOCTYPE and structure.

Topic Types:

Concept	For conceptual information that helps users understand a subject. Uses `<concept>` root element with `<conbody>`.
Task	For procedural information with step-by-step instructions. Uses `<task>` root element with `<taskbody>` and `<steps>`.
Reference	For reference information like API documentation or command syntax. Uses `<reference>` root element with `<refbody>`.
Topic	Generic topic type for content that doesn't fit other categories. Uses `<topic>` root element with `<body>`.

Usage:

1. Run the command from Command Palette
2. Select the topic type
3. Enter a file name (without extension)
4. The file is created in the current folder with `.dita` extension

DITA: Create New Map

Creates a new DITA map file for organizing topics.

Map Structure:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE map PUBLIC "-//OASIS//DTD DITA Map//EN"
 "map.dtd">
<map id="my-map">
    <title>My Map Title</title>
    <topicref href="topic1.dita"/>
    <topicref href="topic2.dita"/>
```

```
</map>
```

Usage:

1. Run the command from Command Palette
2. Enter a file name (without extension)
3. The file is created with `.ditamap` extension

After creation, add `<topicref>` elements to reference your topics.

DITA: Create New Bookmap

Creates a new bookmap for book-oriented publications.

Bookmap Structure:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE bookmap PUBLIC "-//OASIS//DTD DITA
 BookMap//EN" "bookmap.dtd">
<bookmap id="my-bookmap">
    <booktitle>
        <mainbooktitle>My Book Title</
mainbooktitle>
    </booktitle>
    <frontmatter>
        <booklists><toc/></booklists>
    </frontmatter>
    <chapter href="chapter1.dita"/>
    <backmatter>
        <booklists><indexlist/></booklists>
    </backmatter>
</bookmap>
```

Bookmap Elements:

- `<booktitle>` - Book title information
- `<bookmeta>` - Metadata (author, dates, rights)
- `<frontmatter>` - TOC, preface, dedication
- `<chapter>` - Main content chapters
- `<part>` - Groups of chapters
- `<appendix>` - Supplementary content
- `<backmatter>` - Index, glossary

DITA: Initialize DITA Project

Scaffolds a brand-new DITA project in one guided flow: a root map or bookmap, an optional set of starter topics, an optional starter `.ditaval` filter, and the recommended `topics/`, `maps/`, and `images/` folder layout. See Templates and Project Scaffolding on page 105 for the full wizard walkthrough.

Custom Topic Templates

All three file-creation commands above — and the Project Initialization Wizard — honor `ditacraft.templatesPath` when it is set, generating new files from your own template content instead of the built-in boilerplate shown above. See Templates and Project Scaffolding on page 105 for the recognized template file names and placeholder syntax.

File Naming Best Practices

- Use lowercase letters and hyphens (e.g., `getting-started.dita`)
- Avoid spaces and special characters
- Use descriptive names that reflect content
- Keep names concise but meaningful

Chapter 7

Editing and Refactoring Commands

Commands for restructuring content, inserting images and tables, and applying multi-file edits.

Rename Symbol (F2)

Shortcut: F2

Place the cursor on an `id` attribute value, or a single key name inside a `keys="..."` attribute, and press **F2** to rename it and update every `href`, `conref`, `keyref`, and `conkeyref` reference across the workspace. See Refactoring Tools on page 103 for details on how `conkeyref` matches are verified before rewriting.

Move or Rename a File

Drag a `.dita`, `.ditamap`, or `.bookmap` file to a new location in the Explorer, or press **F2** on the file itself, and confirm the prompt to update every inbound reference. See Refactoring Tools on page 103.

DITA: Extract Topic from Section

Place the cursor inside a `<section>` in the active topic to extract it into a new standalone topic file, replacing the original section with an `<xref>`. See Refactoring Tools on page 103.

DITA: Inline Conref

Place the cursor on an element carrying `conref` or `conkeyref` to resolve and splice the referenced content in place, removing the reference attribute. See Refactoring Tools on page 103.

DITA: Insert Image

Browse for an image file and insert a correctly structured `<image>` (optionally wrapped in `<fig>`) at the cursor. See Authoring Productivity Tools on page 109.

DITA: Insert Table

Prompts for table type, column/row counts, and an optional header row, then inserts a `<simpletable>` or CALS `<table>` skeleton at the cursor. See Authoring Productivity Tools on page 109.

DITA: Find and Replace in Files

Searches across the DITA files in your workspace and applies replacements through VS Code's native multi-file Refactor Preview. See Authoring Productivity Tools on page 109.

DITA: Batch Update Metadata

Multi-select files in the DITA Explorer to set or clear a profiling attribute on all of them at once, validated against your subject scheme first. See Authoring Productivity Tools on page 109.

DITA: Edit DITAVAL Conditions

Opens the Visual DITAVAL Condition Editor for the selected `.ditaval` file. See Visual DITAVAL Condition Editor on page 111.

Chapter 8

Navigation Commands

Commands for navigating and visualizing DITA content structure.

DITA: Show Map Visualizer

Opens an interactive tree view showing your DITA map hierarchy.

Features:

- **Hierarchical View:** Displays maps, chapters, and topics in a tree structure
- **Click Navigation:** Click any node to open the corresponding file
- **Real-time Updates:** Tree updates when files are added or removed
- **Status Indicators:** Shows validation status for each file

Usage:

1. Open a `.ditamap` or `.bookmap` file
2. Run the command from Command Palette
3. The visualizer opens in a side panel

Ctrl+Click Navigation

DitaCraft provides smart navigation using Ctrl+Click (Cmd+Click on Mac) on various attributes.

Supported Attributes:

Attribute	Navigation Behavior
`href`	Opens the referenced file
`conref`	Navigates to the content reference target (file#id)
`keyref`	Resolves the key and navigates to its target
`conkeyref`	Resolves the key and navigates to the content reference

Example:

```
<topicref href="topics/intro.dita"/>  <!-- Ctrl
+Click opens intro.dita -->
<p conref="shared.dita#shared/note1"/>  <!-- Ctrl
+Click opens shared.dita at #note1 -->
<ph keyref="product-name"/>  <!-- Ctrl+Click
 resolves key and navigates -->
```

Go to Definition

Press **F12** or right-click and select **Go to Definition** to navigate to referenced content.

This works on the same attributes as Ctrl+Click navigation.

Peek Definition

Press **Alt+F12** to peek at the definition without leaving your current file.

A small inline editor shows the referenced content, allowing you to:

- View the target content
- Make quick edits
- Navigate to the full file

Find All References

Press **Shift+F12** to find all places where the current element or ID is referenced.

Useful for:

- Finding all conrefs to a specific element
- Locating topic references in maps
- Understanding content reuse patterns

Chapter 9

Features Overview

DitaCraft provides a comprehensive set of features for DITA authoring in Visual Studio Code.

Core Features

DitaCraft transforms VS Code into a full-featured DITA authoring environment with the following capabilities:

LSP-Powered IntelliSense	Context-aware auto-completion for 364+ DITA elements, attributes, and attribute values. Hover documentation, document outline, and workspace-wide symbol search (**Ctrl+T**). Powered by a built-in Language Server Protocol implementation.
Smart Navigation	Navigate between files using Ctrl +Click on href, conref, keyref, and conkeyref attributes. Cross-file Go to Definition, Find References, and Rename for IDs and references.
Full DTD Validation	Real-time validation with three engine options: built-in (default, lightweight), TypesXML (full DTD, 100% W3C conformance), and xmllint (external). LSP diagnostics provide instant feedback as you type.
DITAVAL Support	Full IntelliSense for DITAVAL conditional processing files with element completion, attribute suggestions, and action keyword values.
Live Preview	Side-by-side HTML5 preview with bidirectional scroll sync. See your formatted output without running a full build.
DITA-OT Integration	One-click publishing to HTML5, PDF, and other formats using DITA Open Toolkit integration.

Map Visualizer	Interactive tree view showing your DITA map hierarchy with click-to-navigate functionality.
Key Resolution	Full key space support for keyref and conkeyref resolution with improved root map discovery and intelligent key definition lookup.
Cross-Reference Validation	Validates href, conref, keyref, and conkeyref references across your workspace. Catches broken links, missing target files, and undefined keys before publishing.
DITA Rules Engine	43 Schematron-equivalent rules organized in four categories (mandatory, recommendation, authoring, accessibility) enforce DITA best practices and specification requirements. Includes 10 DITA 2.0-specific rules for removed elements and attributes, with automatic version detection.
Profiling and Subject Scheme Validation	Validates profiling attribute values (audience, platform, product, etc.) against subject scheme constraints, ensuring controlled vocabularies are respected.
Activity Bar Views	Dedicated DitaCraft sidebar with three tree views: DITA Explorer (map hierarchy browser), Key Space (defined/undefined/unused keys), and Diagnostics (validation issues grouped by file or severity). Includes file decoration badges for validation status and root map selector in the status bar.
AI Features	Copilot-powered assistance for DITA authoring: restructure maps with a plain-language instruction, explain validation errors, analyze the semantic structure of any DITA element, and discover conref/keyref reuse opportunities. Requires GitHub Copilot (or a configured Anthropic, OpenAI, or Ollama provider). See AI Features on page 93 for details.
Root Map Management	Explicitly set a root map for your project to control key resolution, cross-reference validation, and subject scheme discovery. Status bar indicator shows the current root map. See Root Map Management on page 85 for details.

Localized Diagnostics	All diagnostic messages are localized with support for English and French. The language follows your VS Code display language setting automatically. See Localization (i18n) on page 87 for details.
Refactoring Tools	Rename an ID or key across every usage — with `conkeyref` matches verified against the key space before rewriting — move a topic with automatic reference updates, extract a section into a new topic, and inline a conref/conkeyref back into the referencing element. See Refactoring Tools on page 103 for details.
Templates and Project Scaffolding	Generate new topics, maps, and bookmaps from your own template files instead of the built-in boilerplate, and scaffold a complete new project — root map, starter topics, folder layout — with a guided wizard. See Templates and Project Scaffolding on page 105 for details.
Publishing Profiles and Watch Mode	Save and reuse a publish configuration (transtype, output directory, DITAVAL filter, extra arguments), and automatically republish whenever a watched file changes. See Publishing Workflow Enhancements on page 107 for details.
Authoring Productivity Tools	Insert correctly structured images and tables, and preview multi-file find-and-replace or batch metadata edits with VS Code's native Refactor Preview before anything is written. See Authoring Productivity Tools on page 109 for details.
Visual DITAVAL Condition Editor	Build DITAVAL conditions in a form-based panel with a live preview of the content each condition affects, and see excluded/flagged passages highlighted directly in the editor. See Visual DITAVAL Condition Editor on page 111 for details.

Editor Enhancements

DitaCraft enhances the VS Code editing experience for DITA files:

- **Syntax Highlighting:** DITA-aware XML syntax highlighting
- **Auto-completion:** Context-aware element, attribute, and value suggestions via LSP

- **Hover Documentation:** Hover over any DITA element to see its description and allowed attributes. Hover over keyref/conkeyref to preview resolved content inline
- **Linked Editing:** Edit opening and closing XML tags simultaneously
- **Bracket Matching:** Automatic tag matching and highlighting
- **Folding:** Collapse sections, steps, and other block elements
- **Outline View:** Navigate document structure in the Outline panel
- **Workspace Symbols:** Search for elements across all open files with **Ctrl** +**T**
- **Code Actions:** Quick fixes for missing DOCTYPE, missing ID, missing title, empty elements, duplicate IDs, DITA rules violations (add `@otherrole`, remove deprecated `<indextermref>`, convert `alt` attribute to `<alt>` element, add missing `<alt>`) plus AI-powered fixes (`# Fix with DitaCraft AI`)
- **XML Formatting:** Format DITA documents with proper indentation and inline/block handling

Validation Features

DitaCraft offers multiple validation options:

- **Real-time Validation:** LSP-powered diagnostics appear as you type with smart debouncing (300ms for topics, 1000ms for maps)
- **Multiple Engines:** Choose from built-in (default), TypesXML, or xmllint, with optional RNG (RelaxNG) validation
- **OASIS Catalog Support:** DTD validation via TypesXML with OASIS XML Catalog resolution and parser pool optimization
- **Detailed Diagnostics:** Line-precise error messages with fix suggestions, localized in English and French
- **Code Actions:** Quick fixes for common issues (missing DOCTYPE, IDs, titles) and DITA rules violations
- **Cross-Reference Validation:** Validates href, conref, keyref, and conkeyref targets across files
- **DITA Rules Engine:** 43 Schematron-equivalent rules including 10 DITA 2.0-specific rules
- **Profiling Validation:** Subject scheme-based validation with hierarchy grouping and default value preselection
- **Rate Limiting:** Protection against excessive validation requests

Productivity Features

- **Quick File Creation:** Create topics, maps, and bookmaps from Command Palette
- **Content Reuse:** Full support for conref and keyref mechanisms
- **Cross-file References:** Find all references to an element across your entire workspace
- **Cross-file Rename:** Rename an ID and update all references across files
- **Document Links:** Clickable links for href, conref, keyref, and conkeyref attributes
- **Workspace Support:** Works with single files or multi-root workspaces
- **cSpell Integration:** Auto-prompted spell checking setup with DITA-specific dictionary

- **Activity Bar Views:** Dedicated sidebar with DITA Explorer, Key Space browser, and Diagnostics view for project-level navigation and issue tracking

Chapter 10

IntelliSense

DitaCraft provides LSP-powered IntelliSense for DITA authoring with auto-completion, hover documentation, and symbol navigation.

Overview

DitaCraft includes a built-in Language Server Protocol (LSP) implementation that provides rich editing features for DITA files. The LSP server runs in the background and provides real-time feedback as you type, including auto-completion suggestions, hover documentation, and document symbols.

Auto-Completion

DitaCraft offers context-aware auto-completion for DITA content:

Element Completion	Type `<` to see a list of valid child elements for the current context. The suggestions are filtered based on the parent element's content model, showing only elements that are allowed at the current position. Supports 364+ DITA elements.
Attribute Completion	Inside an opening tag, type a space to see available attributes for the current element. Common attributes like `id`, `href`, and `class` are suggested based on the element type.
Attribute Value Completion	When typing an attribute value (after `=`), DitaCraft suggests valid values from predefined enumerations. For example, the `type` attribute on `<note>` suggests values like `attention`, `caution`, `danger`, etc.
Subject Scheme Completions	When a subject scheme map constrains profiling attributes, auto-completion shows only allowed values with hierarchy grouping (e.g., `Platform > Linux > Ubuntu`). Default values from the scheme are preselected in the completion list.

Hover Documentation

Hover over any DITA element name to see its documentation, including:

- A description of the element's purpose
- Common attributes and their usage
- The element's content model (what children are allowed)

Documentation is available for 142+ DITA elements covering all commonly used topic types, structural elements, and inline elements.

Reference Preview on Hover: Hover over key references and content references to see resolved content inline:

- **keyref hover:** Shows the key's target file, navtitle, short description, and inline content
- **conkeyref hover:** Resolves the key, then extracts and displays a preview of the referenced element content as XML
- **href/conref hover:** Shows target file information and navigation path

Document Symbols

DitaCraft provides document symbols for navigating your DITA content:

- **Outline View:** The VS Code Outline panel shows the structure of your DITA document with sections, steps, tables, and other structural elements.
- **Go to Symbol:** Press **Ctrl+Shift+O** to quickly navigate to any element in the current document.
- **Workspace Symbols:** Press **Ctrl+T** to search for elements across all DITA files in your workspace.

Code Actions and Quick Fixes

DitaCraft provides code actions (quick fixes) for common issues. When the lightbulb icon appears, click it or press **Ctrl+.** to see available fixes:

- **Add Missing DOCTYPE:** Automatically adds the correct DOCTYPE declaration for your topic type
- **Add Missing ID:** Generates a unique ID attribute for the root element
- **Add Missing Title:** Inserts a `<title>` element where required
- **Remove Empty Element:** Cleans up empty elements that have no content
- **Fix Duplicate ID:** Offers to rename a duplicate ID to a unique value
- **Add `@otherrole`:** Inserts the missing `otherrole` attribute when `role="other"` is detected
- **Remove Deprecated `<indextermref>`:** Removes the deprecated element flagged by the DITA Rules Engine
- **Convert `alt` Attribute to `<alt>` Element:** Migrates the deprecated `alt` attribute syntax on `<image>` to the modern `<alt>` child element
- **Add Missing `<alt>`:** Inserts an empty `<alt>` child element inside `<image>` for accessibility compliance

Linked Editing

When you edit an XML tag name, DitaCraft automatically updates the matching opening or closing tag. This feature uses linked editing ranges to keep tag pairs synchronized.

For example, if you change `<section>` to `<example>`, the corresponding `</section>` is automatically updated to `</example>`.

XML Formatting

DitaCraft formats DITA XML documents with proper indentation and element handling:

- **Block Elements:** Structural elements like `<section>`, `<p>`, `<ul>` are formatted with newlines and indentation
- **Inline Elements:** Elements like `<b>`, `<codeph>`, `<xref>` are kept inline with surrounding text
- **Preformatted Content:** `<codeblock>` and `<pre>` content is preserved as-is

Format a document with **Shift+Alt+F** or format a selection with **Ctrl+K Ctrl+F**.

Range formatting (**Ctrl+K Ctrl+F**) correctly scopes edits to the selected lines. When formatting changes the document structure (e.g., collapsing or expanding elements changes line count), DitaCraft safely falls back to replacing the affected range as a whole to prevent data loss.

Chapter 11

Smart Navigation

Navigate between DITA files using Ctrl+Click on reference attributes.

Overview

Smart Navigation allows you to quickly move between related DITA files by clicking on reference attributes. Hold **Ctrl** (or **Cmd** on Mac) and click on any supported attribute value to navigate to the target.

href Navigation

The `href` attribute is used in maps to reference topics and in topics to link to other content.

Examples:

```
<!-- In a ditamap -->
<topicref href="topics/getting-started.dita"/>

<!-- In a topic -->
<xref href="reference-guide.dita#config-settings"/
>
```

Ctrl+Click on the href value opens the referenced file. If the href includes a fragment identifier (#id), the cursor moves to that element.

conref Navigation

Content references (`conref`) pull content from one location to another. Navigation takes you to the source element.

Example:

```
<p conref="shared/warnings.dita#warnings/safety-
note"/>
```

Ctrl+Click navigates to `shared/warnings.dita` and positions the cursor at the element with `id="safety-note"`.

keyref Navigation

Key references use indirect addressing through keys defined in a map. DitaCraft resolves keys using its full DITA 1.3 spec-compliant key space, including scope-qualified keys and multi-hop keyref chains.

Map Definition:

```
<keydef keys="product-name">
    <topicmeta>
```

```
        <keywords><keyword>DitaCraft</keyword></
keywords>
    </topicmeta>
</keydef>

<keydef keys="install-guide" href="topics/
installation.dita"/>

<!-- Scope-qualified keys (keyscope="enterprise"
 on the submap) -->
<keydef keys="install-guide" href="topics/
enterprise-install.dita"/>
```

Usage:

```
<ph keyref="product-name"/>           <!--
 Resolves to "DitaCraft" -->
<xref keyref="install-guide"/>        <!-- Links
 to installation.dita -->
<xref keyref="enterprise.install-guide"/>  <!--
 Scope-qualified lookup -->
```

Ctrl+Click on a keyref:

1. Resolves the key using the full key space (including scope-qualified aliases)
2. Follows any keyref chain steps to reach the final target
3. Navigates to the key's target file or inline definition

Hovering over a `keyref` shows the resolved target, source map location, and any chain steps in the hover tooltip.

conkeyref Navigation

Content key references combine key resolution with content referencing.

Example:

```
<!-- Key definition -->
<keydef keys="shared-content" href="shared/
common.dita"/>

<!-- Usage -->
<note conkeyref="shared-content/warning-note"/>
```

Ctrl+Click resolves `shared-content` to `shared/common.dita`, then navigates to `#warning-note`.

Cross-File Features

DitaCraft provides cross-file navigation and editing via the LSP server:

Go to Definition	Press **F12** or Ctrl+Click on href, conref, keyref, or conkeyref to jump to the target element across files.
Find All References	Press **Shift+F12** on an `id` attribute to find all conref and href references to that element across your workspace.

Rename Symbol	Press **F2** on an `id` attribute to rename it and update all references across files.
Document Links	All href, conref, keyref, and conkeyref values are clickable links. Hover to see the resolved target path.

Navigation Tips

- **Hover Preview:** Hover over a reference to see a preview without navigating
- **Go Back:** Use **Alt+Left Arrow** to return to the previous location
- **Peek Definition:** Use **Alt+F12** to peek without leaving your file
- **Open to Side: Ctrl+Alt+Click** opens the target in a split editor

Chapter 12

Validation

DitaCraft validates DITA files against XML syntax, DTD specifications, cross-references, DITA rules, and profiling constraints in real-time.

Validation Overview

DitaCraft provides comprehensive validation to ensure your DITA content is well-formed, conforms to DITA specifications, uses valid cross-references, follows DITA best practices, and respects profiling constraints. Validation runs automatically as you edit and can also be triggered manually.

What Gets Validated

XML Well-formedness	Checks for proper XML syntax: matching tags, valid attribute values, proper encoding, and correct entity references.
DTD Conformance	Validates elements and attributes against DITA 1.2, 1.3, and 2.0 DTDs. Ensures elements appear in allowed contexts with required attributes.
Content Model	Verifies child elements appear in the correct order and quantity according to the DTD content model.
ID Uniqueness	Checks that ID attributes are unique within a topic.
Cross-Reference Validation	Validates href, conref, keyref, and conkeyref attributes to ensure target files exist, topic and element IDs are present, and keys are defined in the key space. See Cross-Reference Validation on page 67 for details.
DITA Rules Engine	43 Schematron-equivalent rules in five categories (mandatory, recommendation, authoring, accessibility, DITA 2.0 removal) enforce DITA specification requirements and best practices. Includes 10 DITA 2.0-specific rules for removed elements and attributes. Rules are version-aware and include 12 quick fixes. Supports user-defined

	custom regex rules, per-rule severity overrides, and comment-based rule suppression. See DITA Rules Engine on page 69 for details.
Profiling Validation	When a subject scheme map is present, validates that profiling attribute values (audience, platform, product, etc.) use only the allowed values defined by the scheme. See Profiling and Subject Scheme Validation on page 75 for details.

Validation Engines

DitaCraft supports three validation engines, each with different capabilities:

Engine	Pros	Cons
`typesxml`	Full DTD support, 100% W3C conformance, no external dependencies	Slightly slower for very large files
`built-in`	Very fast, lightweight, basic content model checking	Limited DTD support, may miss some errors
`xmllint`	Industry-standard libxml2, excellent performance	Requires external installation

Recommendation: The `built-in` engine (the default) provides a good balance of speed and accuracy for most users. Switch to `typesxml` when you need strict DTD conformance checking.

Additional Validation:

- **OASIS Catalog Support:** The `typesxml` engine uses an OASIS XML Catalog to resolve PUBLIC identifiers to bundled DTDs, with a parser pool (3 instances) for efficient reuse.
- **RNG (RelaxNG) Validation:** Optional RelaxNG schema validation via `salve-annos` and `saxes`. Enable by setting `ditacraft.schemaFormat` to `"rng"` and providing a schema path via `ditacraft.rngSchemaPath`. RNG schemas are compiled to JSON and cached (up to 20 grammars).

LSP-Powered Diagnostics

DitaCraft uses a built-in Language Server Protocol (LSP) implementation to provide real-time diagnostics. The LSP server validates your documents in the background and reports issues instantly as you type, without waiting for a manual save or validation trigger.

The LSP diagnostics cover:

- XML well-formedness (mismatched tags, invalid syntax)
- DITA structure validation (element context, required attributes)
- ID uniqueness within documents

- Cross-reference validation (href, conref, keyref, conkeyref)
- DITA rules (specification compliance, best practices, accessibility)
- Profiling validation (subject scheme constraints)

LSP diagnostics appear with the source label `dita-lsp` in the Problems panel, while client-side validations appear with the `dita` source label. DITA rules diagnostics appear with the source label `dita-rules`.

All diagnostic messages are localized — see Localization (i18n) on page 87 for supported languages.

Diagnostic Levels

Validation issues are reported at three severity levels:

- **Error (Red):** Critical issues that must be fixed. The document is not valid DITA.
- **Warning (Yellow):** Potential issues that should be reviewed. The document may be valid but has problems.
- **Information (Blue):** Suggestions and notes. Not errors, but helpful for improvement.

Viewing Validation Results

Validation results appear in multiple places:

- **Editor Squiggles:** Colored underlines appear on problematic content
- **Problems Panel:** Full list of all diagnostics (**Ctrl+Shift+M**)
- **Status Bar:** Shows error/warning count for the current file
- **File Explorer:** File icons show validation status

Automatic Validation

By default, DitaCraft validates automatically:

- **On Open:** When you open a DITA file
- **On Save:** When you save a file (configurable)
- **On Change:** After you stop typing, with smart debouncing:
  - **Topics:** 300ms debounce for fast feedback
  - **Maps:** 1000ms debounce (maps are more expensive to validate)

  Per-document cancellation ensures only the latest edit triggers validation.

To disable automatic validation, set `ditacraft.autoValidate` to `false` in settings.

Rate Limiting

To prevent performance issues, DitaCraft limits validation requests:

- Maximum 10 requests per second per file
- Excess requests are silently skipped during auto-validation
- Manual validation shows a warning if rate limited

This protects against excessive CPU usage when making rapid edits.

Pipeline Budget

The validation pipeline enforces a configurable time budget (default 30 seconds) to prevent runaway validation on very large or complex files. Before

each major validation phase, the elapsed time is checked; if the budget is exhausted, remaining phases are skipped and the diagnostics collected so far are returned.

The budget protects against:

- Extremely large DITA files that would block the LSP server
- Complex custom regex rules that consume excessive CPU
- Pathological input that triggers worst-case behavior in any validation phase

Individual phases (DITA Rules and Custom Rules) also enforce per-phase timeouts to isolate slow rules from blocking the entire pipeline.

Chapter 13

Live Preview

View your DITA content as formatted HTML5 in a side-by-side preview panel.

Overview

The Live Preview feature renders your DITA content as HTML5 in real-time, allowing you to see how your documentation will look without running a full DITA-OT build.

Shortcut: Ctrl+Shift+H (Windows/Linux) or **Cmd+Shift+H** (Mac)

Preview Features

Live Updates	The preview refreshes automatically as you type. Changes appear within milliseconds, providing instant feedback on your edits.
Bidirectional Scroll Sync	Scrolling in the editor scrolls the preview to the corresponding position, and vice versa. Keep your place when reviewing long documents.
Theme Support	The preview adapts to VS Code's current theme. Light themes show light preview; dark themes show dark preview.
Print Mode	Toggle print mode to see how your content will appear when printed. Useful for reviewing PDF output styling.

Content Rendering

The preview renders the following DITA elements with appropriate styling:

- **Structure:** Sections, paragraphs, titles, and nested content
- **Lists:** Ordered, unordered, and definition lists
- **Tables:** Simple tables and complex tables with spanning
- **Code:** Code blocks and inline code with syntax highlighting
- **Notes:** Note, tip, warning, caution, and danger admonitions
- **Steps:** Task steps with numbering and substeps
- **Images:** Referenced images with alt text

Conref Resolution

The preview resolves content references (conref) and displays the referenced content inline. This lets you see your complete document with all reused content in place.

Example:

```
<p conref="shared/common.dita#common/legal-
notice"/>
```

In the preview, this shows the actual content from the referenced element, not just a placeholder.

Keyref Resolution

Key references are resolved using the current key space. If you have key definitions in your map, the preview shows the resolved values.

Preview Controls

The preview panel includes a toolbar with these controls:

- **Refresh:** Manually refresh the preview
- **Toggle Scroll Sync:** Enable/disable scroll synchronization
- **Toggle Print Mode:** Switch between screen and print styling
- **Open in Browser:** Open the preview in your default browser

Limitations

The live preview is designed for quick content review and has some limitations compared to full DITA-OT output:

- Styling may differ from your DITA-OT configuration
- Some advanced features like relationship tables are not rendered
- Custom plugins and extensions are not applied

For final output review, use **DITA: Publish to HTML5** to generate full DITA-OT output.

Chapter 14

Map Visualizer

View your DITA map structure as an interactive tree diagram.

Overview

The Map Visualizer provides a graphical representation of your DITA map hierarchy. It displays the relationship between maps, chapters, and topics in an easy-to-navigate tree view.

Command: DITA: Show Map Visualizer

Opening the Visualizer

1. Open a `.ditamap` or `.bookmap` file
2. Open Command Palette (**Ctrl+Shift+P**)
3. Type `DITA: Show Map Visualizer`
4. The visualizer opens in a side panel

Tree Structure

The visualizer displays your map hierarchy with different node types:

Map Node	The root of the tree, representing the ditamap or bookmap file.
Chapter/Part Nodes	Major divisions in bookmaps, shown as expandable folders.
Topic References	Individual topic files referenced by the map.
Nested Maps	Submaps referenced via `<mapref>` appear as expandable subtrees.

Navigating the Tree

- **Click:** Select a node and open the corresponding file in the editor
- **Double-click:** Open the file and close the visualizer panel
- **Expand/Collapse:** Click the arrow icons to show or hide child nodes
- **Right-click:** Access context menu with additional options

Visual Indicators

The visualizer uses icons and colors to convey information:

- **File Icons:** Different icons for maps, topics, and other file types
- **Validation Status:** Red indicators show files with errors
- **Missing Files:** Broken references are highlighted

- **Current File:** The currently edited file is highlighted

Real-time Updates

The visualizer updates automatically when:

- You add or remove topicref elements from the map
- You create or delete topic files
- Validation status changes for any file
- You switch between maps

Use Cases

- **Content Audit:** Review your document structure at a glance
- **Navigation:** Quickly jump to any topic in a large documentation set
- **Validation Review:** Identify which files have errors
- **Structure Planning:** Visualize how content is organized

Chapter 15

Key Resolution

Use keys for indirect addressing and content reuse in your DITA projects. DitaCraft implements a fully spec-compliant DITA 1.3 key space construction algorithm including keyref chains, keyscope inheritance, and resolution diagnostics.

Overview

DITA keys provide a powerful mechanism for indirect addressing. Instead of using direct file paths, you define keys in your map and reference them throughout your topics. This enables:

- Easy updates when file locations change
- Conditional content through key scopes
- Centralized management of reusable content
- Simplified localization workflows

Defining Keys

Keys are defined in DITA maps using `<keydef>` elements or the `keys` attribute on `<topicref>`.

Basic Key Definition:

```
<keydef keys="product-name">
    <topicmeta>
        <keywords>
            <keyword>DitaCraft</keyword>
        </keywords>
    </topicmeta>
</keydef>
```

Key with href:

```
<keydef keys="install-guide" href="topics/
installation.dita"/>
```

Key on topicref:

```
<topicref keys="getting-started" href="topics/
getting-started.dita"/>
```

Using keyref

Reference keys using the `keyref` attribute to pull in text content or create links.

Text Replacement:

```
<!-- In your topic -->
<p>Welcome to <ph keyref="product-name"/>!</p>

<!-- Renders as -->
<p>Welcome to DitaCraft!</p>
```

Link Creation:

```
<xref keyref="install-guide">Installation Guide</
xref>
```

Using conkeyref

The `conkeyref` attribute combines key resolution with content referencing. It first resolves the key to a file, then pulls content from a specific element.

```
<!-- Key definition -->
<keydef keys="shared-warnings" href="shared/
warnings.dita"/>

<!-- Usage -->
<note conkeyref="shared-warnings/safety-warning"/>
```

This resolves `shared-warnings` to `shared/warnings.dita`, then pulls the element with `id="safety-warning"`.

Key Space

DitaCraft builds a key space from your root map. The key space includes:

- All key definitions in the root map
- Keys from referenced submaps
- Keys inherited through key scopes

Key Precedence: When the same key is defined multiple times, the first definition wins. Keys defined earlier in the map take precedence.

Key Scopes

Key scopes allow you to define variations of the same content for different contexts.

```
<topicgroup keyscope="admin">
    <keydef keys="audience">

 <topicmeta><keywords><keyword>administrators</
keyword></keywords></topicmeta>
    </keydef>
    <topicref href="admin-guide.dita"/>
</topicgroup>

<topicgroup keyscope="user">
    <keydef keys="audience">
        <topicmeta><keywords><keyword>end users</
keyword></keywords></topicmeta>
    </keydef>
    <topicref href="user-guide.dita"/>
</topicgroup>
```

Keyref Chains

A keyref chain occurs when a key definition itself uses a `keyref` attribute to point to another key, forming a multi-hop chain. DitaCraft follows the full chain to resolve the final target.

Example:

```
<!-- root.ditamap -->
<keydef keys="product" href="topics/product.dita"/
>

<!-- submap.ditamap with @keyscope="enterprise" --
>
<keydef keys="product" keyref="product"/>
```

When resolving `keyref="enterprise.product"`, DitaCraft follows the chain through the scoped alias back to the root definition in `product.dita`.

Important: Circular keyref chains are detected and short-circuited to prevent infinite loops.

Keyscope Inheritance

DitaCraft implements the DITA 1.3 PushDown inheritance algorithm for key scopes. When a map element has a `@keyscope` attribute, its keys are made available under scope-qualified names throughout the parent map.

Scope on a map element:

```
<topicref href="submap.ditamap" keyscope="product-
a"/>
```

All keys in `submap.ditamap` are accessible as `product-a.`*`keyname`* from the parent map.

Scope on a non-map topicref (inline branch):

```
<topicref href="chapter.dita" keyscope="draft">
    <topicref href="draft-notes.dita"/>
</topicref>
```

DitaCraft treats `@keyscope` on a non-map topicref as an inline scope branch: keys defined within that branch are qualified under `draft.*` while still being accessible by their short names within the branch itself.

Multiple scope names: A single element can declare multiple scopes separated by spaces (`keyscope="a b"`). DitaCraft creates qualified aliases under each scope name.

Scope Explosion Protection

Deeply nested key scopes can produce a very large number of qualified key aliases (one per scope combination). DitaCraft enforces a cap of **50,000 key space entries** to prevent excessive memory use and slow builds.

When the cap is reached:

- DitaCraft stops generating additional scope aliases
- A `scopeExplosionWarning` flag is set on the key space
- All keys defined before the cap was reached remain available

In practice, this limit is only reached with extremely large maps that have many deeply nested key scopes. Normal DITA projects are not affected.

Key Provenance

Every key definition in DitaCraft's key space records its **source line number** (1-based) within the map file where it was defined. This information is used by:

- **Go to Definition** — jumps to the exact line of the `<keydef>` element
- The Key Space View — shows the source location in tooltips
- Hover on keyref — displays the definition location alongside the resolved target

For scope-qualified aliases generated from a nested scope, the source line is inherited from the original key definition.

Key Resolution Diagnostics

DitaCraft provides tools for understanding exactly how a key was resolved, which is useful for debugging complex map hierarchies.

When you hover over a `keyref` or `conkeyref` attribute, the hover tooltip shows:

- The resolved target file or inline content
- The key scope context used for resolution
- The source map and line where the key was defined
- Any keyref chain steps that were followed

The underlying `explainKey()` API (used by the LSP server) returns a full `KeyResolutionReport` containing every lookup step, the effective key space, and the complete keyref chain — useful for extension authors building on top of DitaCraft.

DitaCraft Key Support Summary

DitaCraft provides full DITA 1.3 spec-compliant key support:

- **Key Space Construction:** BFS traversal with full cascade algorithm, peer map handling, and fallback key definitions
- **Keyref Chains:** Multi-hop resolution with correct scope prefix tracking
- **Keyscope Inheritance:** PushDown algorithm for map-level scopes and inline branch scopes on non-map topicrefs
- **Scope Explosion Protection:** 50,000-entry cap with warning flag
- **Provenance Tracking:** Source line recorded for every key definition
- **Navigation:** Ctrl+Click on keyref and conkeyref navigates to the key's target with Go to Definition across files
- **Caching:** Key space cached with 5-minute TTL, automatic invalidation on map changes
- **Root Map Selection:** Explicitly set the root map via **DITA: Set Root Map**
- **Validation:** Undefined and missing keys flagged as diagnostics

Chapter 16

DITAVAL Support

DitaCraft provides full editing support for DITAVAL conditional processing files.

Overview

DITAVAL files define conditional processing profiles that control which content is included, excluded, or flagged during DITA publishing. DitaCraft recognizes `.ditaval` files and provides IntelliSense to help you author them efficiently.

What is DITAVAL?

A DITAVAL file is an XML file that specifies filtering and flagging rules for conditional DITA content. It uses attribute-value pairs to control which content appears in the output based on conditions like audience, platform, or product.

Basic DITAVAL Example:

```
<val>
    <prop action="include" att="audience"
 val="admin"/>
    <prop action="exclude" att="audience"
 val="novice"/>
    <prop action="flag" att="platform"
 val="windows">
        <startflag imageref="icons/windows.png">
            <alt-text>Windows</alt-text>
        </startflag>
    </prop>
</val>
```

IntelliSense for DITAVAL

DitaCraft provides the following IntelliSense features for DITAVAL files:

Element Completion	Auto-completion for DITAVAL elements: `<val>`, `<prop>`, `<revprop>`, `<startflag>`, `<endflag>`, and `<alt-text>`.
Attribute Completion	Suggestions for attributes on each element, including `action`, `att`, `val`, and `imageref`.
Attribute Value Completion	Predefined values for the `action` attribute (`include`, `exclude`,

`passthrough`, `flag`) and standard DITA conditional attributes for the `att` attribute (`audience`, `platform`, `product`, `otherprops`, `props`, `rev`).

DITAVAL Elements Reference

Element	Description
`<val>`	Root element of a DITAVAL file
`<prop>`	Defines a filtering or flagging rule based on an attribute-value pair
`<revprop>`	Defines a rule for revision-based filtering or flagging
`<startflag>`	Specifies a flag image or text inserted before flagged content
`<endflag>`	Specifies a flag image or text inserted after flagged content
`<alt-text>`	Alternative text for a flag image

Using DITAVAL with DITA-OT

To apply a DITAVAL profile when publishing, pass it as a filter argument to DITA-OT. In DitaCraft, you can add it to the `ditacraft.ditaOtArgs` setting:

```
"ditacraft.ditaOtArgs": ["--
filter=myprofile.ditaval"]
```

This applies the conditional processing rules from your DITAVAL file during publishing, filtering and flagging content as specified.

Chapter 17

Cross-Reference Validation

DitaCraft validates cross-file references including href, conref, keyref, and conkeyref attributes to catch broken links before publishing.

Overview

Cross-reference validation checks that all references in your DITA content point to valid targets. This includes file paths, topic IDs, element IDs, and key definitions. Broken references are reported as diagnostics so you can fix them before publishing.

Cross-reference validation runs automatically as part of the LSP diagnostics pipeline alongside XML well-formedness and DTD validation.

What Gets Validated

href Attributes	Validates that the target file exists. If the href includes a fragment identifier (e.g., `file.dita#topic_id/element_id`), the topic ID and element ID are also checked. External URLs (`http://`, `https://`) and references with `scope="external"` are skipped.
conref Attributes	Validates that the referenced file exists and that the target topic and element IDs are present in the target file. Same-file conrefs (starting with #) are validated against IDs within the current document.
keyref Attributes	Validates that the referenced key is defined in the key space. If the key has an href target, that target file is also checked.
conkeyref Attributes	Validates the key portion against the key space and, if present, the element ID portion against the target file.

Diagnostic Codes

Cross-reference validation produces the following diagnostic codes:

Code	Severity	Description
DITA-XREF-001	Warning	Target file not found
DITA-XREF-002	Warning	Topic ID not found in target file
DITA-XREF-003	Warning	Element ID not found in target file
DITA-KEY-001	Warning	Undefined key (not in key space)
DITA-KEY-002	Information	Key is defined but has no href target
DITA-KEY-003	Warning	Key target file is missing the referenced element ID

Configuration

Cross-reference validation is enabled by default. To disable it:

```
{
    "ditacraft.crossRefValidationEnabled": false
}
```

When disabled, only XML well-formedness, DTD, and DITA structure validation are performed.

Chapter 18

DITA Rules Engine

DitaCraft includes 43 Schematron-equivalent validation rules organized in five categories to enforce DITA best practices and specification requirements, including 10 DITA 2.0-specific rules, plus support for user-defined custom regex rules.

Overview

The DITA Rules Engine validates your content against a set of rules derived from the DITA specification and authoring best practices. These rules go beyond DTD validation to catch semantic issues such as missing accessibility attributes, deprecated elements, and incorrect attribute usage.

Rules are version-aware: DitaCraft automatically detects your DITA version (from `@DITAArchVersion` or DOCTYPE) and only applies rules relevant to that version.

Rule Categories

Mandatory	Rules required by the DITA specification. These catch constraint violations that would cause processing errors. Examples: `role="other"` requires `@otherrole`, deprecated `<indextermref>` element usage.
Recommendation	Best practice rules recommended by the DITA specification. Examples: adding short descriptions, using proper list structures, following naming conventions.
Authoring	Content quality rules for consistent and well-structured documentation. Examples: title length limits, nesting depth, content organization.
Accessibility	Rules ensuring your content is accessible to all users. Examples: `<image>` elements should have `<alt>` text, deprecated `alt` attribute should be converted to an `<alt>` element.

Key Rules

Some commonly encountered rules include:

Code	Category	Description
`DITA-SCH-001`	Mandatory	`role="other"` requires `@otherrole` attribute
`DITA-SCH-002`	Mandatory	`<note type="other">` requires `@othertype` attribute
`DITA-SCH-003`	Mandatory	`<indextermref>` is deprecated
`DITA-SCH-004`	Mandatory	`collection-type` not allowed on `<reltable>` or `<relcolspec>`
`DITA-SCH-011`	Accessibility	Deprecated `alt` attribute on `<image>` — use `<alt>` element instead
`DITA-SCH-030`	Accessibility	Missing `<alt>` element for `<image>`
`DITA-SCH-040`	Mandatory	Nested `<xref>` not allowed
`DITA-SCH-044`	Recommendation	Deprecated role values (`sample`, `external`)
`DITA-SCH-046`	Recommendation	Elements with `<title>` should have `@id`

Quick Fixes

DitaCraft provides 12 automatic quick fixes via the lightbulb menu (**Ctrl+.**):

- **Add missing DOCTYPE:** Inserts the appropriate DOCTYPE declaration based on root element
- **Add missing ID:** Adds an ID attribute derived from the filename
- **Add missing title:** Inserts a `<title>` element
- **Remove empty element:** Removes elements with no content
- **Rename duplicate ID:** Makes duplicate IDs unique
- **Add `@otherrole`:** Inserts the missing attribute when `role="other"` is detected
- **Remove deprecated `<indextermref>`:** Removes the deprecated element
- **Convert `alt` attribute to `<alt>` element:** Migrates the deprecated attribute syntax to the modern element form
- **Add missing `<alt>` to `<image>`:** Inserts an empty `<alt>` child element for accessibility

- **Sanitize invalid ID:** Fixes invalid ID format by replacing illegal characters and ensuring a valid starting character
- **Insert missing `<booktitle>`:** Adds the required booktitle structure to bookmaps
- **Insert missing `<mainbooktitle>`:** Adds the required mainbooktitle inside an existing booktitle

DITA 2.0 Rules

DitaCraft includes 10 rules specific to DITA 2.0 that detect removed elements and attributes. These rules only apply when the DITA version is detected as 2.0 (via `@DITAArchVersion` or DOCTYPE).

Code	Category	Description
`DITA-SCH-050`	Mandatory	`<boolean>` element removed in DITA 2.0
`DITA-SCH-051`	Mandatory	`<indextermref>` element removed in DITA 2.0
`DITA-SCH-052`	Mandatory	`<object>` removed — use `<audio>`, `<video>`, or `<include>`
`DITA-SCH-053`	Mandatory	Learning specializations removed in DITA 2.0
`DITA-SCH-054`	Accessibility	`<audio>` should have `<fallback>` element
`DITA-SCH-055`	Accessibility	`<video>` should have `<fallback>` element
`DITA-SCH-056`	Mandatory	`@print` attribute removed in DITA 2.0
`DITA-SCH-057`	Mandatory	`@copy-to` attribute removed in DITA 2.0
`DITA-SCH-058`	Mandatory	`@navtitle` attribute removed in DITA 2.0
`DITA-SCH-059`	Mandatory	`@query` attribute removed in DITA 2.0

Version Detection

DitaCraft automatically detects the DITA version of each document using these sources (in priority order):

1. The `ditacraft.ditaVersion` setting (if not set to `"auto"`)
2. The `@DITAArchVersion` attribute on the root element
3. The DOCTYPE public identifier (e.g., `-//OASIS//DTD DITA 2.0 Topic//EN`)
4. Defaults to DITA 1.3 if no version indicator is found

Version-specific rules are only applied when the detected version matches their applicability range.

Custom Regex Rules

In addition to the built-in rules, you can define custom validation rules using regex patterns in a JSON file. Set the `ditacraft.customRulesFile` setting to an absolute path pointing to your rules file.

Each rule supports:

- **id:** A unique identifier (e.g., `CUSTOM-001`)
- **pattern:** A regular expression to match against document content
- **message:** The diagnostic message displayed to the user
- **severity:** One of `error`, `warning`, `information`, `hint`
- **fileTypes:** (Optional) Array of DITA file types to restrict the rule to (topic, concept, task, reference, glossentry, troubleshooting, map, bookmap)

Rules are matched against content with comments and CDATA stripped to avoid false positives. The rules file is automatically reloaded when it changes.

Safety: Custom regex patterns are screened for ReDoS (Regular Expression Denial of Service) vulnerabilities such as nested quantifiers. Patterns that fail the safety check are skipped with a warning diagnostic. Additionally, each rule is limited to 10,000 match iterations and a 2-second timeout to prevent any single rule from blocking the validation pipeline.

Rule Suppression

You can suppress specific rules using:

- **Settings:** Use `ditacraft.validationSeverityOverrides` to set any diagnostic code to `"off"`
- **Inline comments:** Add `<!-- ditacraft-disable CODE -->` to suppress a code from that line until a matching `<!-- ditacraft-enable CODE -->`
- **File-level:** Add `<!-- ditacraft-disable-file CODE -->` anywhere in the file to suppress the code for the entire file

Configuration

Enable or disable the entire rules engine:

```
{
    "ditacraft.ditaRulesEnabled": true
}
```

Select which rule categories to apply:

```
{
    "ditacraft.ditaRulesCategories": [
        "mandatory",
        "recommendation",
        "authoring",
        "accessibility"
    ]
}
```

For example, to run only mandatory and accessibility rules:

```
{
```

```
    "ditacraft.ditaRulesCategories": ["mandatory",
 "accessibility"]
}
```

Chapter 19

Profiling and Subject Scheme Validation

DitaCraft validates profiling attribute values against subject scheme constraints, ensuring controlled vocabularies are respected.

Overview

Subject scheme maps define controlled vocabularies for profiling attributes such as `audience`, `platform`, `product`, and `otherprops`. When a subject scheme is present in your workspace, DitaCraft validates that profiling attribute values use only the allowed values defined by the scheme.

How It Works

1. DitaCraft automatically discovers subject scheme maps in your workspace (or from the explicit root map)
2. The Subject Scheme Service parses the scheme and extracts the controlled values for each attribute, building hierarchy paths for grouped values
3. When you use profiling attributes in your DITA content, their values are checked against the allowed values
4. Invalid values are reported with a diagnostic that lists the valid options
5. Auto-completion for profiling attributes shows only allowed values with hierarchy grouping (e.g., `Platform > Linux > Ubuntu`) and default value preselection

Subject Hierarchy Grouping

When subject scheme definitions use nested `<subjectdef>` elements to define a hierarchy, DitaCraft displays this hierarchy in auto-completion suggestions. For example:

```
<subjectdef keys="os">
    <subjectdef keys="linux">
        <subjectdef keys="ubuntu"/>
        <subjectdef keys="fedora"/>
    </subjectdef>
    <subjectdef keys="windows"/>
</subjectdef>
```

When completing `platform` attribute values, you will see suggestions like:

- `ubuntu` — shown as `os > linux > ubuntu`
- `fedora` — shown as `os > linux > fedora`
- `windows` — shown as `os > windows`

If the scheme defines a default value, that value is preselected in the completion list.

Validated Attributes

The following profiling attributes are validated when controlled by a subject scheme:

- `audience`
- `platform`
- `product`
- `otherprops`
- `props`
- `deliveryTarget`

Attributes that are not constrained by a subject scheme are not validated — you can use any value freely.

Subject Scheme Example

A subject scheme map that constrains the `platform` attribute:

```
<subjectScheme>
    <subjectdef keys="platform-values">
        <subjectdef keys="windows"/>
        <subjectdef keys="linux"/>
        <subjectdef keys="macos"/>
    </subjectdef>
    <enumerationdef>
        <attributedef name="platform"/>
        <subjectdef keyref="platform-values"/>
    </enumerationdef>
</subjectScheme>
```

With this scheme active, using `platform="android"` would produce a warning because `android` is not in the allowed values list.

Diagnostic Codes

Code	Severity	Description
`DITA-PROF-001`	Warning	Attribute value not in controlled vocabulary. The diagnostic message lists the valid values.

Configuration

Subject scheme validation is enabled by default. To disable it:

```
{
    "ditacraft.subjectSchemeValidationEnabled":
 false
}
```

Chapter 20

Activity Bar Views

DitaCraft provides a dedicated sidebar in the VS Code Activity Bar with three tree views for project navigation, key management, and diagnostics.

Overview

The DitaCraft Activity Bar adds a dedicated icon to the VS Code sidebar. Clicking it reveals three specialized views that give you an at-a-glance overview of your DITA project:

- **DITA Explorer:** Browse all workspace maps with expandable hierarchy and type-specific icons
- **Key Space:** View all defined, undefined, and unused keys with usage locations
- **Diagnostics:** Centralized DITA validation issues grouped by file or severity

To open the DitaCraft sidebar, click the DitaCraft icon in the Activity Bar on the left side of VS Code.

Additionally, the VS Code status bar shows the current **root map** status. Click it to set or change the root map for key resolution and cross-reference validation. See Root Map Management on page 85 for details.

Views at a Glance

DITA Explorer	Shows all `.ditamap` and `.bookmap` files in your workspace as expandable trees. Each map displays its topic hierarchy with icons for chapters, topics, keydefs, and other element types. Click any item to open the referenced file. Right-click for context menu actions such as validate, publish, or show the map visualizer.
Key Space	Lists all DITA keys organized into three groups: **Defined Keys** (used and defined), **Undefined Keys** (used but not defined in any map), and **Unused Keys** (defined but never referenced). Expand any key to see where it is used across your workspace.
Diagnostics	Aggregates all DITA-related validation issues from the Problems

panel. Switch between grouping by file or by severity. Click any issue to navigate directly to its location in the editor.

Welcome Content

When a view has no data to display, DitaCraft shows helpful welcome messages:

- **DITA Explorer (no maps):** Suggests creating a new map or bookmap
- **DITA Explorer (no folder):** Suggests opening a folder containing DITA files
- **Key Space (empty):** Explains that keys will appear once maps with key definitions are found
- **Diagnostics (clean):** Confirms that no DITA validation issues were found

Refreshing Views

All three views auto-refresh when relevant files change. You can also manually refresh each view using the refresh button in the view title bar or the corresponding commands:

- **DITA: Refresh DITA Explorer**
- **DITA: Refresh Key Space**
- **DITA: Refresh Diagnostics**
- **DITA: Set Root Map** — Set an explicit root map for the project
- **DITA: Clear Root Map (Auto-Discover)** — Return to automatic root map discovery

Chapter 21

DITA Explorer

Browse all workspace DITA maps with an expandable hierarchy tree in the Activity Bar sidebar.

Overview

The DITA Explorer is the primary view in the DitaCraft Activity Bar. It discovers all `.ditamap` and `.bookmap` files in your workspace and displays each one as an expandable tree showing its full topic hierarchy.

Command: DITA: Refresh DITA Explorer

Opening the DITA Explorer

1. Click the DitaCraft icon in the Activity Bar (left sidebar)
2. The DITA Explorer view appears at the top of the sidebar
3. All workspace maps are listed as root nodes

Tree Structure

Each map is displayed as a tree with type-specific icons:

Map	Root-level ditamap or bookmap files, shown with a hierarchy icon. These are always the top-level nodes.
Chapter	Chapter elements from bookmaps, shown with a book icon.
Part	Part elements from bookmaps, shown with a folder icon.
Appendix	Appendix elements, shown with a file-add icon.
Topic Reference	Standard `<topicref>` elements, shown with a file icon.
Key Definition	`<keydef>` elements, shown with a key icon.
Missing File	References to files that do not exist are shown with a warning icon.

Nested maps referenced via `<mapref>` are automatically expanded as subtrees. Circular references are detected and marked.

Navigation and Actions

- **Click:** Open the referenced file in the editor

- **Expand/Collapse:** Click the arrow to show or hide child elements
- **Context Menu:** Right-click a node for additional actions:
  - **Validate** - Run validation on the file
  - **Publish** - Publish the file (maps only)
  - **Show Map Visualizer** - Open the Map Visualizer panel (maps only)

Each item shows the `href` value as a description next to the label, and a tooltip with type, href, keys, and status information.

Validation Badges

Files in the DITA Explorer show validation status badges from the File Decoration Provider:

- **Error badge:** Red indicator with error count for files with validation errors
- **Warning badge:** Yellow indicator with warning count for files with warnings only
- **No badge:** Clean files with no validation issues

Badges update automatically as diagnostics change.

Auto-Refresh

The DITA Explorer watches for changes to `.ditamap`, `.bookmap`, and `.dita` files. When files are created, modified, or deleted, the tree refreshes automatically after a 500ms debounce period to avoid excessive rebuilds during batch operations.

To manually refresh, click the refresh button in the view title bar or run **DITA: Refresh DITA Explorer** from the Command Palette.

Chapter 22

Key Space View

Browse all DITA keys in your workspace organized by status: defined, undefined, and unused.

Overview

The Key Space view in the DitaCraft Activity Bar shows every DITA key across your workspace, grouped by its resolution status. This helps you identify missing key definitions, find unused keys, and navigate to key usages.

Command: DITA: Refresh Key Space

Key Groups

Keys are organized into three groups:

Defined Keys	Keys that are both defined in a map (via `<keydef>` or `<topicref keys="...">`) and referenced somewhere in your DITA content (via `keyref` or `conkeyref` attributes). These are healthy keys that resolve correctly.
Undefined Keys	Keys that are referenced in your content but have no matching definition in any workspace map. These typically indicate a missing `<keydef>` element or a typo in a `keyref` attribute. Shown with a warning icon.
Unused Keys	Keys that are defined in a map but never referenced anywhere in your DITA content. These may be candidates for cleanup or may indicate content that needs to use the key.

Navigating Keys and Usages

- **Click a key:** Navigate to the map file where the key is defined
- **Expand a key:** See all files that reference this key via `keyref` or `conkeyref`
- **Click a usage:** Navigate directly to the usage location in the editor, with the reference highlighted

Each key item shows additional details:

- **Description:** The target filename for keys with an `href`, or "(inline)" for keys with inline content
- **Tooltip:** Key name, target file, source map, and scope
- **Usage description:** Reference type (`keyref` or `conkeyref`) and line number

How Keys Are Collected

DitaCraft builds the key space using a DITA 1.3 spec-compliant BFS traversal:

1. Starting from the root map, traverses the map hierarchy in breadth-first order
2. Extracts `<keydef>` and key-bearing `<topicref>` elements; first definition wins (DITA precedence rule)
3. Applies `@keyscope` PushDown inheritance: keys within a scoped branch get scope-qualified aliases (e.g., `product-a.install-guide`)
4. Follows keyref chains across scopes to resolve multi-hop references
5. Records provenance (source line) for every key definition
6. Enforces a 50,000-entry cap to prevent scope explosion from deeply nested scopes
7. Scans all `.dita`, `.ditamap`, and `.bookmap` files (up to 500) for `keyref` and `conkeyref` attributes
8. Compares definitions against usages to classify each key into Defined / Undefined / Unused

For `conkeyref` values like `conkeyref="my-key/element-id"`, only the key name before the slash is extracted for classification purposes.

Auto-Refresh

The Key Space view watches for changes to `.ditamap` and `.bookmap` files. When map files are created, modified, or deleted, the view refreshes after a 1000ms debounce period. Key space resolution can be expensive for large workspaces, so the debounce is longer than other views.

To manually refresh, click the refresh button in the view title bar or run **DITA: Refresh Key Space** from the Command Palette.

Use Cases

- **Key Audit:** Review which keys are defined, used, and missing
- **Cleanup:** Identify unused keys that can be removed from maps
- **Debugging:** Find undefined keys causing unresolved references
- **Navigation:** Quickly jump to key definitions or usages across files

Chapter 23

Diagnostics View

View all DITA validation issues in a centralized tree with grouping by file or severity.

Overview

The Diagnostics view in the DitaCraft Activity Bar aggregates all DITA-related validation issues from across your workspace. Unlike the built-in Problems panel, this view filters specifically for DITA diagnostics and offers two grouping modes for easier triage.

Command: DITA: Refresh Diagnostics

Grouping Modes

Switch between two ways of organizing diagnostics using the view title bar buttons:

Group by File	Issues are grouped under their parent file. Each file group shows the total issue count and a summary of errors and warnings. The file icon color reflects the highest severity (red for errors, yellow for warnings). Click the **Group by File** button in the view title bar to activate.
Group by Severity	Issues are grouped into four severity categories: Errors, Warnings, Information, and Hints. Each group shows its count. Errors and warnings are expanded by default; information and hints are collapsed. Click the **Group by Severity** button in the view title bar to activate.

Diagnostic Items

Each diagnostic item displays:

- **Severity icon:** Error (red circle), Warning (yellow triangle), Information (blue info), or Hint (outline circle)
- **Message:** The validation issue description
- **Line number:** Shown as a description (e.g., "line 12")
- **Tooltip:** Full message, source (e.g., `dita`, `dita-lsp`, `ditacraft-keys`), and line number

Click any diagnostic item to open the file and jump to the exact location of the issue.

Diagnostic Sources

The Diagnostics view only shows issues from DITA-related sources:

- `dita` - Client-side DITA validation
- `dita-lsp` - Language Server validation (XML, structure, IDs)
- `ditacraft-keys` - Key reference diagnostics

Diagnostics from other extensions (e.g., spell checkers, generic XML linters) are excluded to keep the view focused on DITA-specific issues.

Auto-Refresh

The Diagnostics view refreshes automatically whenever validation results change (e.g., after editing a file, saving, or opening a new document). Updates are debounced at 300ms to batch rapid changes.

To manually refresh, click the refresh button in the view title bar or run **DITA: Refresh Diagnostics** from the Command Palette.

Clean State

When no DITA validation issues are found, the view shows a welcome message confirming that your workspace is clean. This is the ideal state after resolving all validation issues.

Chapter 24

Root Map Management

Explicitly set a root map for your project to control key resolution, cross-reference validation, and subject scheme discovery.

Overview

DitaCraft uses a root map to build the key space, resolve cross-references, and discover subject scheme constraints. By default, DitaCraft auto-discovers root maps in your workspace. Starting with v0.6.2, you can explicitly set a root map for precise control over key resolution and validation behavior.

Automatic Discovery

When no explicit root map is set, DitaCraft automatically discovers maps in your workspace using the following strategy:

1. Scans for `.ditamap` and `.bookmap` files
2. Selects the most likely root map based on structure and references
3. Builds the key space from the discovered map

Auto-discovery works well for projects with a single root map. For complex projects with multiple maps, setting an explicit root map is recommended.

Setting a Root Map

To explicitly set the root map:

1. Open the Command Palette (**Ctrl+Shift+P**)
2. Run **DITA: Set Root Map**
3. Select the desired map file from the list of available maps

The selected root map is stored in your workspace settings (`ditacraft.rootMap`) and persists across sessions.

Clearing the Root Map

To return to automatic discovery:

1. Open the Command Palette (**Ctrl+Shift+P**)
2. Run **DITA: Clear Root Map (Auto-Discover)**

This removes the explicit root map setting and DitaCraft reverts to auto-discovery mode.

Status Bar Indicator

The current root map status is displayed in the VS Code status bar at the bottom of the window. It shows:

- **Map name:** When an explicit root map is set, the filename is displayed
- **Auto:** When DitaCraft is using automatic map discovery

Click the status bar item to quickly change the root map.

Configuration

The root map setting can also be configured directly in `settings.json`:

```
{
    "ditacraft.rootMap": "maps/master.ditamap"
}
```

The path is relative to the workspace root. Leave empty or omit for automatic discovery.

What the Root Map Affects

The root map determines the scope for:

- **Key Space:** All `keyref` and `conkeyref` resolution uses keys defined in the root map hierarchy
- **Cross-Reference Validation:** Key-based references are validated against the root map's key space
- **Subject Scheme Discovery:** Subject scheme maps referenced from the root map define profiling constraints
- **DITA Explorer:** The root map hierarchy is used for the tree view
- **Key Space View:** Shows defined, undefined, and unused keys from the root map

Chapter 25

Localization (i18n)

DitaCraft supports localized diagnostic messages, currently available in English and French.

Overview

Starting with version 0.6.2, DitaCraft localizes diagnostic messages produced by the DITA Rules Engine, cross-reference validation, profiling validation, and structural validation. The language is detected automatically from your VS Code display language setting.

Supported Languages

Language	Locale Code	Status
English	`en`	Default / Fallback
French	`fr`	Full translation (67 messages)

How It Works

1. When the LSP server starts, it reads the locale from VS Code's `InitializeParams.locale` setting
2. The corresponding message bundle (`en.json` or `fr.json`) is loaded
3. All diagnostic messages use the localized strings with placeholder interpolation
4. If a translation is missing for a key, the English fallback is used automatically

What Is Localized

The following diagnostic messages are localized:

- **XML validation:** Syntax errors, well-formedness issues
- **DITA structure validation:** Missing DOCTYPE, missing ID, missing title, empty elements, duplicate IDs
- **Cross-reference validation:** Broken links, missing files, undefined keys
- **DITA Rules Engine:** All 43 rules (mandatory, recommendation, authoring, accessibility)
- **Profiling validation:** Invalid attribute values
- **DTD validation:** Catalog-based DTD conformance messages

Changing the Language

DitaCraft follows your VS Code display language. To change it:

1. Open the Command Palette (**Ctrl+Shift+P**)
2. Type `Configure Display Language`
3. Select your preferred language (e.g., `fr` for French)
4. Restart VS Code

DitaCraft diagnostics will automatically appear in the selected language on the next server start.

Chapter 26

Guide Validation (DITA-OT)

DitaCraft can validate your entire guide by running DITA-OT against the root map, producing a detailed WebView report with filtering, search, and export.

Overview

The **DITA: Validate Entire Guide Using DITA-OT** command performs end-to-end validation of your documentation project by invoking DITA-OT against the configured root map. Unlike file-level validation (which checks XML syntax and DITA rules), guide validation runs the full DITA-OT processing pipeline to detect cross-file issues such as broken references, missing images, unresolved keys, and build errors that only surface during publishing.

Prerequisites

Guide validation requires:

- **DITA-OT installed and configured** — Set the path via `ditacraft.ditaOtPath` or the **DITA: Configure DITA-OT Path** command.
- **Root map configured** — Set via **DITA: Set Root Map** or the `ditacraft.rootMap` setting. The root map must exist in the workspace.

If either prerequisite is missing, the command displays an error with a quick action to configure the missing item.

How to Run

1. Open the Command Palette (**Ctrl+Shift+P**).
2. Type `DITA: Validate Entire Guide` and select the command.
3. A progress notification appears while DITA-OT runs.
4. When complete, a WebView report panel opens beside the editor.

Only one guide validation can run at a time. If you trigger the command while validation is in progress, a notification informs you to wait.

Transtype Selection

DitaCraft automatically selects the best transtype for validation:

- **dita** (preferred) — An identity transform available in DITA-OT 3.6+ that runs the full preprocessing pipeline without generating output files. This is the fastest option for pure validation.

- **html5** (fallback) — Used when the `dita` transtype is not available. Generates HTML5 output in a temporary directory that is automatically cleaned up.

Validation Report

The report WebView provides a rich interface for reviewing issues:

Summary Cards	Shows error, warning, info counts, and number of affected files at a glance.
Severity Filtering	Click filter buttons to show only errors, warnings, or info messages.
Search	Type in the search box to filter issues by message, error code, file name, description, or module.
Grouping	Group issues by file (default), severity, or DITA-OT module (Core, Processing, Transform, Indexing, PDF, XEP).
File Navigation	Click any file link to open the file at the exact line and column in the editor.
Error Code Tooltips	Hover over an error code badge to see the human-readable description from the DITA-OT error catalog (160+ codes).
Export	Export the full report as a JSON file for archiving or CI integration.
Rerun	Click the **Rerun Validation** button to re-execute without leaving the report panel.

Problems Panel Integration

In addition to the WebView report, guide validation populates the VS Code Problems panel with all errors and warnings. Issues appear with the source label `DITA-OT` and include the DITA-OT error code (e.g., `DOTX010E`). Click any diagnostic to navigate to the source location.

DITA-OT Error Code Catalog

DitaCraft includes a built-in catalog of 160+ DITA-OT error codes with human-readable descriptions and module categorization. Error codes follow the format `[PREFIX][NUMBER][SEVERITY]` where:

- **DOTA** — Core transformation errors
- **DOTJ** — Processing pipeline (Java) errors
- **DOTX** — XSLT transform errors
- **INDX** — Index processing errors
- **PDFJ/PDFX** — PDF generation errors
- **XEPJ** — XEP FO processor errors

The severity suffix indicates: **E** = Error, **W** = Warning, **F** = Fatal, **I** = Info.

> DitaCraft recognizes both the legacy format (`[DOTJ013E][ERROR]`) and the modern severity-first format used by DITA-OT 3.x and 4.x (`[ERROR] [DOTJ013E]`), ensuring error codes are correctly extracted regardless of your DITA-OT version.

Chapter 27

AI Features

DitaCraft integrates GitHub Copilot and other LLM providers to assist with DITA map restructuring, error explanation, element analysis, and content reuse discovery.

Overview

DitaCraft's AI features connect to an LLM provider — GitHub Copilot by default — and layer context-aware DITA intelligence on top of plain chat. All AI commands use the current document's DITA context (topic hierarchy, key space, validation diagnostics), not just raw text.

Supported providers, tried in order:

1. **GitHub Copilot** — integrated; no API key required when Copilot is installed.
2. **Anthropic (Claude)** — requires an API key stored via **DitaCraft: Configure AI**.
3. **OpenAI (GPT-4o)** — requires an API key stored via **DitaCraft: Configure AI**.
4. **Ollama** — local inference; requires Ollama running on `localhost:11434` and `ditacraft.ai.provider.ollama.enabled` set to `true`.

Use **DitaCraft: Configure AI** from the Command Palette to view provider status and store API keys securely via VS Code SecretStorage.

Configure AI

Run **Command Palette** > **DitaCraft: Configure AI** to open the AI Settings panel.

The panel shows:

- Live status for each provider (available / not configured / unavailable)
- Input fields for Anthropic and OpenAI API keys. Keys are stored in VS Code SecretStorage (OS keychain integration) — never in plain-text settings or workspace files.
- AI mode selector (`auto`, `copilot-only`, `byok-only`, or `local-only`)

Tip: With `auto` mode (the default), DitaCraft tries each provider in priority order and falls back automatically if one is unavailable.

@ditacraft Chat Participant

DitaCraft registers a Copilot Chat participant named **@ditacraft**. Open the Copilot Chat panel (**Ctrl+Alt+I**) and prefix your message with `@ditacraft` to access the following slash commands.

/restructure

Proposes a restructured DITA map based on a plain-language intention.

Usage: `@ditacraft /restructure <intention>`

Example: `@ditacraft /restructure Group topics by audience: admin first, then end user`

DitaCraft builds a DITA context snapshot from the active `.ditamap`, sends it to the LLM, and streams back valid DITA XML. The response is validated by the LSP before being presented. The active tab must be a `.ditamap` file.

/validate

Explains a DITA validation diagnostic at the cursor position in plain language.

Usage: `@ditacraft /validate [optional question]`

Example: `@ditacraft /validate why does DITA require unique IDs?`

DitaCraft reads the first diagnostic on the current line, includes the surrounding XML context, and asks the LLM to explain the error and suggest a fix. If no diagnostics are present, the response confirms the file is valid.

/explain

Explains the semantic structure and purpose of the selected DITA element.

Usage: `@ditacraft /explain [focus instruction]`

Example: `@ditacraft /explain focus on accessibility best practices`

Select an XML element in the editor before running this command. The selection is sent to the LLM inside a code fence; any focus instruction you

	provide is sent separately as context (not embedded inside the XML).
/suggest-reuse	Identifies content reuse opportunities in the current DITA map using `conref` and `keyref` patterns. **Usage:** `@ditacraft /suggest-reuse [focus instruction]` **Example:** `@ditacraft /suggest-reuse focus on installation topics only` DitaCraft analyzes the map structure and topic content, then returns a list of concrete reuse suggestions with code examples. The active tab must be a `.ditamap` file.

F2: Restructure Map Command

The **DitaCraft: Restructure Map with AI** command provides a full diff-and-apply workflow for restructuring a DITA map.

How to use:

1. Open a `.ditamap` file (it must be the active tab).
2. Open the Command Palette and run **DitaCraft: Restructure Map with AI**. Alternatively, right-click the file in the Explorer and choose **Restructure Map with AI**.
3. Enter your restructuring intention in the InputBox that appears.
4. DitaCraft opens a **diff editor** showing the current map (left) and the proposed restructure (right).
5. A dialog asks **Apply restructured map?** — click **Apply** to save or **Cancel** to discard.

All original `href` attributes are preserved. The change is undoable with **Ctrl+Z**.

F3: AI Quick Fix

For supported diagnostic codes, DitaCraft offers an **# Fix with DitaCraft AI** code action alongside the built-in quick fixes.

To use: click the lightbulb icon or press **Ctrl+.** on a flagged element, then select **# Fix with DitaCraft AI**.

Supported diagnostic codes:

Code	Error type
`DITA-CM-001/002/003`	Invalid child element (content model violation)
`DITA-XREF-001`	Missing or broken `href` target file
`DITA-STRUCT-003/004/005`	Missing required attribute or structural violation

Code	Error type
`DITA-STRUCT-008`	Missing `topicref` target
`DITA-XREF-003/004`	Broken `conref` or cross-reference
`DITA-DTD-001`, `DITA-RNG-001`	DTD or RelaxNG schema violation

Tip: The AI quick fix is context-aware: it reads the surrounding XML before proposing the fix, so the result respects your content model.

F4: AI Completion

When `ditacraft.ai.completion.enabled` is `true` (default), DitaCraft appends AI-suggested elements to the standard LSP completion list.

AI completions are identified by the **(AI)** suffix and the `DitaCraft AI` detail label. They appear below the native LSP completions (sorted last). The AI suggestion is generated with a 500 ms timeout budget — if the LLM does not respond in time, only native completions are shown.

To disable AI completions, set `ditacraft.ai.completion.enabled` to `false` in Settings.

Resilience and Fallback

DitaCraft uses a **circuit breaker** to prevent cascading failures when an LLM provider is unavailable:

- After 3 consecutive failures within 5 minutes, the provider's circuit opens and no further calls are attempted for 10 minutes.
- After the cooldown, one probe request is sent (HALF_OPEN state). If it succeeds, the circuit closes; if not, the cooldown resets.
- Cancelled requests (user pressed Escape) do not count as failures.

When the active provider's circuit is open, DitaCraft automatically falls back to the next available provider in the cascade (Copilot # Anthropic # OpenAI # Ollama).

Metrics and Telemetry

When `ditacraft.ai.telemetry.enabled` is `true`, DitaCraft logs AI call metrics to the **DitaCraft** output channel in the format:

```
[AI] # validate via copilot 1842ms ~340tok
```

Each line shows: command name, provider used, latency in milliseconds, and estimated token count. Failed calls show # instead of #.

Metrics are retained in memory (capped at 1,000 entries) and cleared on extension restart. They are never sent to any remote service.

AI Settings Reference

Setting	Type	Default	Description
`ditacraft.ai.mode`	string	`auto`	Provider cascade mode: `auto`, `copilot-only`, `byok-`

Setting	Type	Default	Description
			`only` (Anthropic or OpenAI only), or `local-only` (Ollama only)
`ditacraft.ai.completion.enabled`	boolean	`true`	Show AI-suggested completions alongside LSP completions
`ditacraft.ai.provider.ollama.enabled`	boolean	`true`	Enable Ollama local inference provider
`ditacraft.ai.provider.ollama.baseUrl`	string	`http://localhost:11434`	Base URL for the Ollama server
`ditacraft.ai.provider.ollama.model`	string	`llama3`	Ollama model name to use
`ditacraft.ai.telemetry.enabled`	boolean	`false`	Log AI call metrics (provider, latency, tokens) to the output channel

CAUTION: Setting `ditacraft.ai.mode` to `local-only` while `ditacraft.ai.provider.ollama.enabled` is `false` will produce a configuration conflict error on startup. Use `byok-only` if you want to use only your Anthropic or OpenAI key without Copilot or Ollama.

Chapter 28

MCP Server Integration

DitaCraft exposes its DITA intelligence — validation, key space resolution, context snapshots, and map analysis — to external AI agents via the Model Context Protocol (MCP).

Overview

The **MCP server** lets AI coding agents like opencode, Claude Desktop, Cursor, and Continue read and query your DITA workspace. It runs as a standalone Node.js process with no VS Code dependency — agents spawn it directly via stdio.

The server exposes **6 tools** and **3 resources**:

Tool	Description
`dita_validate`	Validate a DITA file or XML fragment and return diagnostics with error counts and severity
`dita_context_snapshot`	Token-budgeted map snapshot for LLM prompt injection (Levels 1/2/3)
`dita_key_space`	List all defined keys and their resolved targets with scope and provenance filters
`dita_map_structure`	Return map topic hierarchy as JSON, tree text, or CSV
`dita_resolve_reference`	Resolve href, keyref, conref, or conkeyref to target file and element
`dita_explain_key`	Step-by-step key resolution trace with scope and keyref chain details

Resources: `dita://workspace/maps`, `dita://workspace/diagnostics`, `dita://workspace/keys`

Building the MCP Server

Run the standalone build command from the DitaCraft project root:

```
npm run build-standalone
# # dist/mcp-server.js  (3.2 MB)
# # dist/lsp-server.js  (2.0 MB)
```

Both bundles are self-contained — no `node_modules` needed at runtime.

Using with opencode

Add the DitaCraft MCP server to your opencode configuration:

```
// ~/.config/opencode/opencode.json
{
  "mcpServers": {
    "ditacraft": {
      "command": "node",
      "args": ["/path/to/ditacraft/dist/mcp-
server.js"],
      "env": {
        "WORKSPACE": "/home/user/projects/my-dita-
docs"
      }
    }
  }
}
```

Then use opencode to interact with your DITA workspace:

```
> @ditacraft validate topics/intro.dita
> @ditacraft show me the map structure as a tree
> @ditacraft explain why keyref install-guide
 isn't resolving
> @ditacraft what diagnostics does the workspace
 have?
```

Using with Claude Desktop

Add the server to your Claude Desktop config:

```
// macOS: ~/Library/Application Support/Claude/
claude_desktop_config.json
// Windows: %APPDATA%\Claude
\claude_desktop_config.json
{
  "mcpServers": {
    "ditacraft": {
      "command": "node",
      "args": ["/absolute/path/to/dist/mcp-
server.js"],
      "env": { "WORKSPACE": "/path/to/dita/
project" }
    }
  }
}
```

Standalone LSP Server

DitaCraft also ships a standalone LSP server bundle for embedding in other Node.js projects:

```
node dist/lsp-server.js --stdio
```

Set the `DITACRAFT_EXTENSION_ROOT` environment variable to point to the directory containing the `dtds/` folder. The LSP server uses stdin/stdout for JSON-RPC communication.

Embed programmatically:

```
const { spawn } = require('child_process');
const server = spawn('node', ['lsp-server.js', '--
stdio'], {
    env: { DITACRAFT_EXTENSION_ROOT: __dirname },
});
// Communicate via server.stdin / server.stdout
```

Workspace Security

The MCP server enforces workspace isolation:

- All file paths are resolved relative to the `WORKSPACE` environment variable
- Path traversal attacks (`../../` beyond 8 levels) are rejected
- HTTP URLs, UNC paths, and null bytes are rejected
- No document content is ever transmitted to external services

Tip: The MCP server uses stdio transport by default, which means all communication stays local to the machine. No network configuration is required.

Chapter 29

Refactoring Tools

DitaCraft rewrites every reference to an ID, key, or moved file across your whole workspace — including `conkeyref` matches, which are now verified against the key space before they are touched.

Overview

DITA content grows fragile as it grows large: a single renamed ID or moved file can silently break dozens of `conref`, `keyref`, and `conkeyref` pointers elsewhere in the workspace. DitaCraft's refactoring tools rewrite the reference graph for you, so structural edits stay safe even in large, multi-map projects.

Rename ID or Key

Place the cursor on an `id` attribute value, or on a single key name inside a `keys="..."` attribute, and press **F2** (or right-click and choose **Rename Symbol**). DitaCraft updates every matching `href`, `conref`, `keyref`, and `conkeyref` across the workspace in one atomic edit.

Renaming a key now verifies every candidate `conkeyref` match against the key space before rewriting it — a match on element-ID text alone is no longer enough. A reference that looks like it points at the renamed key but cannot be proven to resolve there is left untouched and logged instead of silently rewritten, eliminating false-positive rewrites in workspaces that reuse the same ID or key name in more than one scope.

Tip: Rename Symbol shows a preview of every file that will change before you confirm — nothing is written until you accept.

Move Topic with Reference Updates

Move or rename a `.dita`, `.ditamap`, or `.bookmap` file the normal VS Code way — drag it in the Explorer, or use **F2** on the file itself — and DitaCraft rewrites every inbound `href` and `conref` that pointed at its old path, in every file across the workspace that referenced it.

VS Code shows a confirmation prompt (**Update references to this file?**) before applying the edits, so you can review or skip the update on any individual move.

Extract Topic from Section

Select — or simply place the cursor inside — a `<section>` in the active topic and run **DITA: Extract Topic from Section**. DitaCraft:

1. Creates a new standalone topic file from the section's content.

2. Matches the new file's root element to the *source* topic's type — a `<concept>` section produces a new `<concept>`, a `<reference>` section produces a new `<reference>`, and anything else falls back to a generic `<topic>`.
3. Replaces the original section with an `<xref>` pointing at the new topic.

Important: A `<taskbody>` does not allow `<section>` as a child at all — DITA tasks use `<context>`, `<steps>`, and `<result>` instead. Running this command inside a task topic is blocked with a clear message rather than producing structurally invalid content.

Inline Conref

The reverse of content reuse: place the cursor on (or inside) an element that carries a `conref` or `conkeyref` attribute and run **DITA: Inline Conref**. DitaCraft resolves the reference, splices the target element's content into the current location, and removes the reference attribute — stripping any nested descendant `id` values from the spliced-in content so the result never introduces a duplicate-ID violation.

The edit applies immediately to the active file, without a workspace-wide preview — unlike Find & Replace or Batch Metadata Update, this is a single, cursor-scoped change. Undo with **Ctrl+Z** if you change your mind.

Chapter 30

Templates and Project Scaffolding

Start a new DITA project — or a new topic — from your own house style instead of DitaCraft's generic generated structure.

Custom Topic Templates

By default, **DITA: Create New Topic**, **DITA: Create New Map**, and **DITA: Create New Bookmap** generate minimal boilerplate. Set `ditacraft.templatesPath` to a directory of your own template files to override that boilerplate with your team's standard structure, boilerplate notices, or starter content.

Recognized file names (matched by topic/map type):

Template file	Used for
`topic.dita`	Generic topic
`concept.dita`	Concept topic
`task.dita`	Task topic
`reference.dita`	Reference topic
`map.ditamap`	DITA map
`bookmap.bookmap`	Bookmap

Template files support `{{placeholder}}` substitution — for example `{{title}}`, `{{id}}`, `{{author}}`, and `{{date}}` — filled in from the same prompts the built-in generators already ask (file name, title, and so on).

Tip: `ditacraft.templatesPath` accepts the `${workspaceFolder}` variable, so a templates directory checked into the project itself (for example `.dita-templates/`) works the same for every contributor.

If no template file matches the requested type — or `ditacraft.templatesPath` is not set at all — DitaCraft falls back to its built-in generator, so file creation always works even with zero configuration.

Project Initialization Wizard

Run **DITA: Initialize DITA Project** to scaffold a brand-new DITA project in one guided flow, instead of creating a root map and its topics one command at a time:

1. **Root type** — a plain DITA map, or a book-structured bookmap (front/back matter, chapters).

2. **Title** — the project or book title, written into the root map.
3. **Root file name** — defaults to `main` for a map, or `user-guide` for a bookmap.
4. **Starter topics** — pick any combination of Concept, Task, Reference, and generic Topic to scaffold alongside the root map. Skipping this step is fine — it leaves you with just a root map, and more topics can always be added later with **DITA: Create New Topic**.

You are also offered an optional starter `.ditaval` filter file, and the wizard creates the recommended `topics/`, `maps/`, and `images/` folder layout for the new project.

The wizard reuses the same file-generation logic as **DITA: Create New Topic**/**Map**/ **Bookmap** — including any custom templates configured via `ditacraft.templatesPath` above.

Chapter 31

Publishing Workflow Enhancements

Save a publish configuration once and reuse it, or let DitaCraft republish automatically every time you save.

Publishing Profiles

A **publishing profile** is a saved combination of transtype (output format), output directory, DITAVAL filter, and extra DITA-OT arguments — so you don't have to re-enter the same publish configuration every time. Run **DITA: Manage Publishing Profiles** from the Command Palette to create, edit, delete, or select the active profile.

Profile fields:

Field	Description
Name	A label you choose, shown in the profile picker
Transtype	Output format — `html5`, `pdf`, `xhtml`, `epub`, `htmlhelp`, `markdown`, or any transtype your DITA-OT installation supports
Output directory	Optional; supports `${workspaceFolder}`
DITAVAL path	Optional filter file, also supports `${workspaceFolder}`
Additional arguments	Extra `-D` flags or other DITA-OT command-line options

Profiles are stored in the `ditacraft.publishingProfiles` workspace setting — commit `.vscode/settings.json` to share the same profiles with your whole team. DitaCraft remembers the last-used profile and pre-selects it the next time you publish.

Watch Mode

Run **DITA: Start Watch Mode** to have DitaCraft automatically re-run a full publish every time a watched DITA file changes — no more manually re-triggering **DITA: Publish** after every edit while reviewing formatted output. Stop it with **DITA: Stop Watch Mode**.

What gets published on each change, in priority order:

1. The file you started watch mode on (for example, by right-clicking a map in the DITA Explorer).

2. Otherwise, the map configured via the `ditacraft.rootMap` setting, if one is set.
3. Otherwise, the active editor's `.dita`, `.ditamap`, or `.bookmap` file.

The format and output directory come from your last-used publishing profile when one exists, falling back to a plain HTML5 publish otherwise.

A single status bar item shows the current state — **Watching**, **Publishing…**, a transient **Published** confirmation, or a persistent **Publish failed** until the next successful run. Publish output still goes through the same DITA-OT build output channel and Problems panel a manual publish uses.

Important: Watch Mode re-runs the *full* publish on every change — it is not incremental. DITA-OT itself has no first-class incremental build mode, so large projects will see the same publish time on every save that a manual publish would take.

Chapter 32

Authoring Productivity Tools

Insert correctly structured images and tables, and preview multi-file edits before anything is written.

Insert Image

Run **DITA: Insert Image** to browse for an image file and insert a correctly formed `<image>` element at the cursor — wrapped in a `<fig><title>`…`</title>`…`</fig>` skeleton when you give it a caption, or inserted bare otherwise.

Picking an image from outside the workspace (for example your OS file picker's Downloads folder) offers to copy it into an `images/` folder next to the current topic, rather than leaving you with a reference to a file the topic can't portably point at. Optional width, height, or scale attributes can be set on the inserted element as well.

Tip: DitaCraft does not check whether `<fig>` or `<image>` is valid at the cursor's exact position — if the inserted markup lands somewhere the content model disallows it, the existing DTD validation flags it immediately, the same as hand-typing invalid markup would.

Insert Table

Run **DITA: Insert Table** to insert a correctly structured table skeleton, eliminating the most common table authoring mistake — mismatched `<entry>` counts against `<colspec>`. You are prompted for:

1. Table type — `<simpletable>` or a full CALS `<table>`/`<tgroup>`.
2. Column and row counts.
3. Whether to include a header row.
4. A table title — offered only for the CALS variant, since `<simpletable>` has no `<title>` child in DITA's content model.

This is an insertion wizard, not a visual grid editor — it produces the skeleton markup with the right column and entry counts already matched up, ready for you to fill in.

Multi-File Find and Replace

Run **DITA: Find and Replace in Files** to search across the DITA files in your workspace with literal text, case sensitivity, whole-word, or regular expression matching, then replace every match in one operation.

Matches are computed by the LSP server and applied as a workspace edit through VS Code's native multi-file **Refactor Preview** — the same review UI used for a cross-file rename — so you see every file and every match

before anything is written, and can uncheck individual changes you don't want applied.

Batch Metadata Update

Multi-select a set of files in the **DITA Explorer** (Ctrl-click or Shift-click to select several, then right-click), choose **DITA: Batch Update Metadata**, pick a profiling attribute (`audience`, `platform`, `product`, `otherprops`, `props`, or `deliveryTarget`), and enter a value to set — or leave it empty to remove the attribute — on every selected file's root element at once.

Each candidate value is validated server-side against your subject scheme's controlled values before anything is written, so a batch edit can't introduce a `DITA-PROF-001` violation across many files simultaneously. Like Find and Replace, every change goes through VS Code's native multi-file Refactor Preview before it's applied.

Chapter 33

Visual DITAVAL Condition Editor

Build and preview DITAVAL conditional-processing profiles with a form-based editor instead of hand-writing XML.

Overview

While DITAVAL Support on page 65 covers IntelliSense for authoring `.ditaval` files directly, the **Visual DITAVAL Condition Editor** is a dedicated panel for building and reviewing conditions without writing XML by hand. Run **DITA: Edit DITAVAL Conditions** on a `.ditaval` file — from the DITA Explorer, the file Explorer's context menu, or the active editor — to open it.

Editing Conditions

The panel lists every `<prop>` condition in the file as a form row — attribute (`audience`, `platform`, `product`, and so on), value, and action (`include`, `exclude`, `passthrough`, or `flag`) — instead of raw attribute-value XML. Add, edit, reorder, or remove conditions from the form, and the underlying `.ditaval` file is kept in sync.

Live Preview with Conditions Applied

The editor renders the content the DITAVAL profile currently resolves to, so you can see the effect of a condition change immediately rather than publishing to check it. Toggle individual conditions on or off to compare included and excluded content side by side.

Condition Highlighting in the Editor

With a `.ditaval` file active in the condition editor, content in open DITA topics that matches an `exclude` or `flag` condition is highlighted directly in the text editor — so you can see exactly which passages a given profile affects without leaving your topic files.

Part III

Configuration

Topics:

Chapter 34

Settings Overview

Configure DitaCraft to match your workflow and preferences.

Accessing Settings

DitaCraft settings can be accessed in several ways:

1. Open VS Code Settings (**Ctrl+,** or **Cmd+,**)
2. Search for `ditacraft`
3. Or edit `settings.json` directly

Settings Levels

VS Code supports multiple settings levels:

User Settings	Apply to all VS Code instances. Stored in your user profile.
Workspace Settings	Apply only to the current workspace. Stored in `.vscode/settings.json`.
Folder Settings	Apply to a specific folder in a multi-root workspace.

Workspace settings override user settings, allowing project-specific configuration.

Settings Categories

DitaCraft settings are organized into these categories:

Category	Description
General	DITA-OT path and basic configuration
Validation	Validation engine and auto-validation settings
Publishing	Output directory and publishing options
Preview	Live preview behavior and styling

Quick Settings Reference

Setting	Default	Description
`ditacraft.ditaOtPath`	`""`	Path to DITA-OT installation
`ditacraft.validationEngine`	`"typesxml"`	Validation engine to use
`ditacraft.autoValidate`	`true`	Enable automatic validation
`ditacraft.outputDirectory`	`"out"`	Output directory for publishing
`ditacraft.previewScrollSync`	`true`	Enable preview scroll sync
`ditacraft.ditaRulesEnabled`	`true`	Enable DITA Rules Engine (43 rules)
`ditacraft.crossRefValidationEnabled`	`true`	Enable cross-reference validation
`ditacraft.ditaVersion`	`"auto"`	Override DITA version detection
`ditacraft.rootMap`	`""`	Explicit root map path

Example settings.json

```
{
    "ditacraft.ditaOtPath": "C:/DITA-OT-4.2.1",
    "ditacraft.validationEngine": "typesxml",
    "ditacraft.autoValidate": true,
    "ditacraft.outputDirectory": "out",
    "ditacraft.previewScrollSync": true,
    "ditacraft.ditaRulesEnabled": true,
    "ditacraft.ditaRulesCategories": ["mandatory",
 "recommendation", "authoring", "accessibility"],
    "ditacraft.crossRefValidationEnabled": true,
    "ditacraft.ditaVersion": "auto",
    "ditacraft.rootMap": ""
}
```

Chapter 35

General Settings

Configure DITA-OT path and basic DitaCraft settings.

ditacraft.ditaOtPath

Type: String

Default: `""` (empty)

The path to your DITA Open Toolkit installation directory. This setting is required for publishing features.

Example Values:

- Windows: `"C:/DITA-OT-4.2.1"`
- macOS: `"/Users/username/dita-ot-4.2.1"`
- Linux: `"/opt/dita-ot-4.2.1"`

Tip: Use forward slashes (/) in paths, even on Windows. VS Code handles path conversion automatically.

How to Set:

1. Run **DITA: Configure DITA-OT Path** from Command Palette, or
2. Set the value manually in settings.json

Verification:

DitaCraft verifies the path by checking for:

- `bin/dita` (Unix) or `bin/dita.bat` (Windows)
- `lib/` directory with required JAR files

Java Configuration

DITA-OT requires Java 11 or higher. DitaCraft uses the following to locate Java:

1. `JAVA_HOME` environment variable
2. Java in system PATH

If Java is not found, publishing will fail with a "Java not found" error.

Setting JAVA_HOME:

```
# Windows (PowerShell)
$env:JAVA_HOME = "C:\Program Files\Java\jdk-17"

# macOS/Linux
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk
```

Workspace Detection

DitaCraft automatically detects workspace context:

- **Single Folder:** Uses the folder as the workspace root
- **Multi-root Workspace:** Uses the folder containing the current file
- **No Workspace:** Uses the directory of the current file

This affects relative path resolution for href, conref, and key definitions.

Chapter 36

Validation Settings

Configure validation engine, automatic validation, cross-reference checking, DITA rules, profiling validation, severity overrides, custom rules, and large file optimization.

ditacraft.validationEngine

Type: String (enum)

Default: `"built-in"`

Options:

`"built-in"`	Lightweight validation with content model checking. Fast and suitable for most DITA authoring workflows. The default engine.
`"typesxml"`	Full DTD validation using TypesXML parser. Provides 100% W3C conformance with complete DTD support. Best for strict compliance checking.
`"xmllint"`	External validation using libxml2's xmllint tool. Requires xmllint to be installed on your system. Provides excellent performance and accuracy.

Choosing an Engine:

Use Case	Recommended Engine
General DITA authoring	`built-in` (default)
Strict DTD conformance required	`typesxml` or `xmllint`
Very large files (1000+ elements)	`built-in` or `xmllint`
Quick syntax checking only	`built-in`

ditacraft.autoValidate

Type: Boolean

Default: `true`

When enabled, DitaCraft validates DITA files automatically:

- When a file is opened
- When a file is saved

- After typing stops, with smart debouncing (300ms for topics, 1000ms for maps)

Set to `false` to disable automatic validation. You can still validate manually using **Ctrl+Shift+V**.

When to Disable:

- Working with very large files that cause performance issues
- Editing files that are intentionally incomplete
- When you prefer manual validation control

ditacraft.maxNumberOfProblems

Type: Number

Default: `100`

The maximum number of diagnostic problems reported per file. Increase this value if you work with very large files and need to see all issues. Decrease it if the Problems panel becomes overwhelming.

ditacraft.ditaRulesEnabled

Type: Boolean

Default: `true`

Enable or disable the DITA Rules Engine. When enabled, DitaCraft validates your content against 43 Schematron-equivalent rules covering specification compliance, best practices, and accessibility (including 10 DITA 2.0-specific rules). Set to `false` to disable all DITA rules validation.

ditacraft.ditaRulesCategories

Type: Array of strings

Default: `["mandatory", "recommendation", "authoring", "accessibility"]`

Select which DITA rule categories to apply. Available categories:

- `"mandatory"` — Rules required by the DITA specification
- `"recommendation"` — Best practices recommended by the DITA specification
- `"authoring"` — Content quality and consistency rules
- `"accessibility"` — Accessibility compliance rules

For example, to run only mandatory and accessibility rules:

```
{
    "ditacraft.ditaRulesCategories": ["mandatory",
 "accessibility"]
}
```

ditacraft.crossRefValidationEnabled

Type: Boolean

Default: `true`

Enable or disable cross-reference validation. When enabled, DitaCraft checks that href, conref, keyref, and conkeyref attributes point to valid targets. Set

to `false` if you are working with incomplete content where references are expected to be unresolved.

ditacraft.subjectSchemeValidationEnabled

Type: Boolean

Default: `true`

Enable or disable subject scheme validation. When enabled and a subject scheme map is present in the workspace, DitaCraft validates that profiling attribute values (audience, platform, product, etc.) match the controlled vocabulary defined by the scheme. Set to `false` to allow any profiling attribute values.

ditacraft.ditaVersion

Type: String (enum)

Default: `"auto"`

Options: `"auto"`, `"1.0"`, `"1.1"`, `"1.2"`, `"1.3"`, `"2.0"`

Override the auto-detected DITA version for validation. When set to `"auto"` (default), DitaCraft detects the version from `@DITAArchVersion` or DOCTYPE. Set an explicit version to force version-specific rules (e.g., `"2.0"` to enable DITA 2.0 rules for all documents).

ditacraft.schemaFormat

Type: String (enum)

Default: `"dtd"`

Options: `"dtd"`, `"rng"`

Schema format for validation. The default `"dtd"` uses standard DTD validation. Set to `"rng"` to enable RelaxNG schema validation using `salve-annos`. Requires `ditacraft.rngSchemaPath` to be set.

ditacraft.rngSchemaPath

Type: String

Default: `""`

Path to the directory containing RNG schema files (e.g., a DITA-OT's schema directory). Required when `ditacraft.schemaFormat` is set to `"rng"`. RNG schemas are compiled to JSON and cached for performance (up to 20 grammars).

ditacraft.rootMap

Type: String

Default: `""`

Explicit root map path (relative to workspace root). Leave empty for automatic root map discovery. Set via the **DITA: Set Root Map** command or directly in settings.

The root map determines the key space for keyref/conkeyref resolution, cross-reference validation scope, and subject scheme discovery. See Root Map Management on page 85 for details.

Rate Limiting

DitaCraft includes built-in rate limiting to prevent performance issues:

- **Limit:** 10 validation requests per second per file
- **Behavior:** Excess requests are silently skipped during auto-validation
- **Manual Validation:** Shows a warning if rate limited

Rate limiting cannot be configured but ensures stable performance during rapid editing.

ditacraft.validationSeverityOverrides

Type: Object

Default: `{}`

Override the severity of any diagnostic code. Map diagnostic codes to `"error"`, `"warning"`, `"information"`, `"hint"`, or `"off"` (to suppress entirely).

```
{
    "ditacraft.validationSeverityOverrides": {
        "DITA-SCH-001": "hint",
        "DITA-ID-002": "off",
        "DITA-STRUCT-005": "error"
    }
}
```

ditacraft.customRulesFile

Type: String

Default: `""`

Absolute path to a JSON file defining custom regex validation rules. Each rule specifies a regex pattern, message, severity, and optional file type filter. Rules are re-loaded automatically when the file changes.

```
{
    "rules": [
        {
            "id": "CUSTOM-001",
            "pattern": "<draft-comment\\b",
            "message": "Draft comments should be
 removed before publishing",
            "severity": "warning",
            "fileTypes": ["topic", "concept",
 "task"]
        }
    ]
}
```

ditacraft.largeFileThresholdKB

Type: Number

Default: `500`

Files larger than this threshold (in KB) skip heavy validation phases (cross-references, DITA rules, profiling, custom rules, etc.) for performance. Set to

`0` to disable the optimization and always run all phases. A `DITA-PERF-001` informational diagnostic is shown when phases are skipped.

Comment-Based Rule Suppression

You can suppress specific diagnostic codes inline using XML comments:

- `<!-- ditacraft-disable CODE -->` — Suppress a code from this point until re-enabled
- `<!-- ditacraft-enable CODE -->` — Re-enable a previously suppressed code
- `<!-- ditacraft-disable-file CODE -->` — Suppress a code for the entire file

Multiple codes can be specified, separated by spaces: `<!-- ditacraft-disable DITA-SCH-001 DITA-ID-002 -->`

Setting Up xmllint

To use the `xmllint` engine, install libxml2:

Windows:

1. Download from zlatkovic.com
2. Add the bin directory to your PATH

macOS:

```
brew install libxml2
```

Linux (Debian/Ubuntu):

```
sudo apt-get install libxml2-utils
```

Verification:

```
xmllint --version
```

Chapter 37

Publishing Settings

Configure output directory and DITA-OT publishing options.

ditacraft.outputDirectory

Type: String

Default: `"out"`

The directory where published output is generated. Can be an absolute path or relative to the workspace root.

Examples:

- `"out"` - Creates `{workspace}/out/`
- `"build/output"` - Creates `{workspace}/build/output/`
- `"C:/Documentation/Output"` - Uses absolute path

Output Structure:

Within the output directory, DitaCraft creates subdirectories for each format:

```
out/
### html5/
#   ### index.html
#   ### topics/
### pdf/
#   ### document.pdf
### xhtml/
    ### ...
```

Default Publishing Format

DitaCraft supports publishing to multiple formats via DITA-OT:

Format	Description	Requirements
html5	Modern responsive HTML5	None (built-in)
pdf	PDF via Apache FOP	FOP (included in DITA-OT)
xhtml	XHTML 1.0 Transitional	None (built-in)
eclipsehelp	Eclipse Help System	None (built-in)
htmlhelp	Microsoft HTML Help	HTML Help Compiler
markdown	Markdown output	None (built-in)

DITA-OT Build Parameters

DitaCraft passes the following parameters to DITA-OT during publishing:

Parameter	Value
`-i`	Input file (current map or topic)
`-f`	Output format (html5, pdf, etc.)
`-o`	Output directory

For advanced DITA-OT configuration, use a `build.xml` or `plugin.xml` file in your project.

Clean Output

By default, publishing overwrites existing output in the target directory. To ensure a clean build:

1. Delete the output directory manually, or
2. Use a different output directory for each build

Viewing Published Output

After publishing completes:

- **HTML5:** Open `out/html5/index.html` in a browser
- **PDF:** Open the generated PDF file in a PDF reader
- **VS Code:** Right-click HTML files and select "Open with Live Server"

Chapter 38

Preview Settings

Configure live preview behavior and appearance.

ditacraft.previewScrollSync

Type: Boolean

Default: `true`

Enables bidirectional scroll synchronization between the editor and preview panel.

Behavior when enabled:

- Scrolling in the editor scrolls the preview to the corresponding position
- Scrolling in the preview scrolls the editor to match
- Cursor position in editor highlights the corresponding preview element

When to Disable:

- Working with very long documents where sync causes performance issues
- When you prefer independent scrolling of editor and preview
- When editing and viewing different sections simultaneously

Theme Integration

The preview automatically adapts to VS Code's current theme:

- **Light Themes:** White background, dark text
- **Dark Themes:** Dark background, light text
- **High Contrast:** Enhanced contrast for accessibility

Preview styling follows VS Code's color theme to provide a consistent experience.

Preview Refresh Behavior

The preview refreshes automatically with the following characteristics:

- **Debounce:** 300ms delay after typing stops
- **Preservation:** Scroll position is preserved during refresh
- **Efficiency:** Only changed content is re-rendered when possible

Preview Panel Controls

The preview panel toolbar provides these controls:

Refresh Button	Manually refresh the preview content.

Scroll Sync Toggle	Enable or disable scroll synchronization.
Print Mode Toggle	Switch between screen and print styling. Print mode shows how content will appear when printed.
Open in Browser	Open the preview in your default web browser for full-screen viewing.

Content Resolution

The preview resolves the following content types:

Content Type	Resolution Behavior
conref	Fetches and displays referenced content inline
keyref	Resolves key to display value or creates link
conkeyref	Resolves key then fetches referenced content
images	Displays images from relative or absolute paths

Note: If a reference cannot be resolved, the preview shows a placeholder indicating the missing content.

Known Limitations

The live preview has some limitations compared to full DITA-OT output:

- Custom DITA-OT plugins are not applied
- Relationship tables are not rendered
- Some advanced table features may display differently
- PDF-specific formatting is not shown

For final output review, use the publishing commands to generate full DITA-OT output.

Appendix
A

Keyboard Shortcuts

Quick reference for all DitaCraft keyboard shortcuts.

Overview

DitaCraft provides keyboard shortcuts for common operations. This reference lists shortcuts for both Windows/Linux and macOS.

Validation

Windows/Linux	Mac	Command	Description
Ctrl+Shift+V	**Cmd+Shift+V**	DITA: Validate Current File	Validate the active DITA file against XML syntax and DTD

Publishing

Windows/Linux	Mac	Command	Description
Ctrl+Shift+B	**Cmd+Shift+B**	DITA: Publish (Select Format)	Publish with format selection dialog
Ctrl+Shift+H	**Cmd+Shift+H**	DITA: Preview HTML5	Open live HTML5 preview panel

Navigation

Windows/Linux	Mac	Action	Description
Ctrl+Click	**Cmd+Click**	Go to Definition	Navigate to href, conref, keyref, or conkeyref target
Ctrl+Alt+Click	**Cmd+Option +Click**	Open to Side	Open referenced file in split editor
F12	**F12**	Go to Definition	Navigate to the definition of the reference under cursor
Alt+F12	**Option+F12**	Peek Definition	Peek at definition inline without

Windows/Linux	Mac	Action	Description
			leaving current file
Shift+F12	**Shift+F12**	Find All References	Find all places where the current element is referenced
Alt+Left Arrow	**Ctrl+- (minus)**	Go Back	Return to previous location after navigation
Alt+Right Arrow	**Ctrl+Shift+- (minus)**	Go Forward	Move forward in navigation history

VS Code General Shortcuts

These VS Code shortcuts are useful when working with DITA files:

Windows/Linux	Mac	Action	Description
Ctrl+Shift+P	**Cmd+Shift+P**	Command Palette	Open Command Palette to access all DitaCraft commands
Ctrl+P	**Cmd+P**	Quick Open	Quickly open files by name
Ctrl+Shift+M	**Cmd+Shift+M**	Problems Panel	Show/hide validation errors and warnings
Ctrl+Shift+O	**Cmd+Shift+O**	Go to Symbol	Navigate to elements within the current file
Ctrl+G	**Ctrl+G**	Go to Line	Jump to a specific line number
Ctrl+/	**Cmd+/**	Toggle Comment	Comment/ uncomment XML content
Ctrl+Shift+[	**Cmd+Option+[**	Fold Region	Collapse the current XML element
Ctrl+Shift+]	**Cmd+Option+]**	Unfold Region	Expand the current XML element
Ctrl+K Ctrl+0	**Cmd+K Cmd+0**	Fold All	Collapse all foldable regions
Ctrl+K Ctrl+J	**Cmd+K Cmd+J**	Unfold All	Expand all folded regions

Windows/Linux	Mac	Action	Description
Ctrl+S	**Cmd+S**	Save	Save file (triggers auto-validation if enabled)
Ctrl+Z	**Cmd+Z**	Undo	Undo last edit
Ctrl+Shift+Z	**Cmd+Shift+Z**	Redo	Redo last undone edit

XML Editing Shortcuts

Windows/Linux	Mac	Action	Description
Ctrl+Space	**Ctrl+Space**	Trigger Suggestions	Show element and attribute auto-completion
Ctrl+Shift +Space	**Cmd+Shift +Space**	Parameter Hints	Show attribute value suggestions
Ctrl+.	**Cmd+.**	Quick Fix	Show available quick fixes for errors
F2	**F2**	Rename Symbol	Rename element ID across files
Ctrl+D	**Cmd+D**	Add Selection	Select next occurrence of current word
Ctrl+Shift+L	**Cmd+Shift+L**	Select All Occurrences	Select all occurrences of current word
Alt+Up/Down	**Option+Up/ Down**	Move Line	Move current line up or down
Shift+Alt+Up/ Down	**Shift+Option +Up/Down**	Copy Line	Duplicate current line above or below
Ctrl+Shift+K	**Cmd+Shift+K**	Delete Line	Delete the current line

Customizing Shortcuts

To customize keyboard shortcuts:

1. Press **Ctrl+K Ctrl+S** (Windows/Linux) or **Cmd+K Cmd+S** (Mac) to open Keyboard Shortcuts
2. Search for the command (e.g., "DITA: Validate")
3. Click the pencil icon to assign a new shortcut
4. Press your desired key combination

You can also edit `keybindings.json` directly for advanced customization.

Quick Cheat Sheet

Most frequently used DitaCraft shortcuts:

Action	**Windows/Linux**	**Mac**
Validate File	**Ctrl+Shift+V**	**Cmd+Shift+V**
Quick Fix (Code Actions)	**Ctrl+.**	**Cmd+.**
Publish	**Ctrl+Shift+B**	**Cmd+Shift+B**
Preview HTML5	**Ctrl+Shift+H**	**Cmd+Shift+H**
Navigate to Reference	**Ctrl+Click**	**Cmd+Click**
Auto-Complete	**Ctrl+Space**	**Ctrl+Space**
Hover Info	Mouse hover / **Ctrl+K Ctrl+I**	Mouse hover / **Cmd+K Cmd+I**
Command Palette	**Ctrl+Shift+P**	**Cmd+Shift+P**
Problems Panel	**Ctrl+Shift+M**	**Cmd+Shift+M**
Format Document	**Shift+Alt+F**	**Shift+Option+F**

Appendix
B

Auto-validation

A DitaCraft feature that automatically validates DITA files when they are opened, saved, or modified. Controlled by the `ditacraft.autoValidate` setting.

Related reference

Validation Settings on page 119

Appendix
C

Bookmap

A specialized DITA map designed for book-oriented publications. Bookmaps support book-specific elements like chapters, parts, appendixes, frontmatter, and backmatter. File extension: `.bookmap`.

Related reference

File Creation Commands on page 31

Appendix
D

Built-in Engine

A lightweight validation engine bundled with DitaCraft that provides fast XML syntax checking and basic content model validation. Faster than TypesXML but with limited DTD support.

Related reference

Validation Settings on page 119

Appendix
E

OASIS XML Catalog

A standard mechanism for mapping PUBLIC identifiers (such as DOCTYPE declarations) to local file paths. DitaCraft uses an OASIS XML Catalog to resolve DITA DTD references to bundled DTD files, enabling offline validation without network access.

Appendix
F

Code Action

A quick fix or automated refactoring offered by the Language Server Protocol when a diagnostic issue is detected. In DitaCraft, code actions appear via the lightbulb icon or Ctrl+. shortcut and include fixes such as adding missing DOCTYPE, fixing duplicate IDs, adding accessibility attributes, and removing deprecated elements.

Related concepts

IntelliSense on page 45

DITA Rules Engine on page 69

Appendix
G

conkeyref

A DITA attribute that combines key resolution with content referencing. The key is first resolved to a file path, then the element ID is used to pull in specific content. Format: `conkeyref="keyname/elementid"`.

Related concepts

Key Resolution on page 61

Smart Navigation on page 49

Appendix
H

conref

Content reference. A DITA mechanism for reusing content by referencing an element in another file. The referenced content is pulled in during processing. Format: `conref="file.dita#topicid/elementid"`.

Related concepts

Smart Navigation on page 49

Appendix
I

Concept

A DITA topic type designed for conceptual information that helps users understand a subject. Concepts explain "what" and "why" rather than "how". Uses the `<concept>` root element with `<conbody>`.

Related reference

File Creation Commands on page 31

Appendix
J

Cross-Reference Validation

A validation feature that checks href, conref, keyref, and conkeyref attributes to ensure they point to valid targets. Cross-reference validation catches broken links, missing files, undefined keys, and invalid element IDs before publishing.

Related concepts

Cross-Reference Validation on page 67

Related reference

Validation Settings on page 119

Appendix
K

DITA

Darwin Information Typing Architecture. An XML-based standard for authoring, producing, and delivering technical documentation. DITA uses specialized topic types and maps to organize modular, reusable content.

Related concepts

Introduction to DitaCraft on page 15

Appendix
L

DITA 2.0

The latest major version of the DITA specification, which removes several deprecated elements (`<boolean>`, `<indextermref>`, `<object>`, learning specializations) and attributes (`@print`, `@copy-to`, `@navtitle`, `@query`), while adding new multimedia elements (`<audio>`, `<video>`, `<fallback>`). DitaCraft includes 10 DITA 2.0-specific validation rules.

Appendix
M

DITA-OT

DITA Open Toolkit. An open-source publishing engine that transforms DITA content into various output formats including HTML5, PDF, EPUB, and more. Required for DitaCraft publishing features.

Related reference

Publishing Commands on page 29

General Settings on page 117

Appendix
N

DITA Rules Engine

A validation engine that applies 18 Schematron-equivalent rules to enforce DITA specification requirements and best practices. Rules are organized in four categories: mandatory, recommendation, authoring, and accessibility. The engine is version-aware and only applies rules relevant to the detected DITA version.

Related concepts

DITA Rules Engine on page 69

Related reference

Validation Settings on page 119

Appendix
O

DITA Map

An XML file that defines the structure and organization of DITA topics. Maps specify which topics to include, their hierarchy, and relationships. File extension: `.ditamap`.

Related concepts

Map Visualizer on page 59

Related reference

File Creation Commands on page 31

Appendix P

DTD

Document Type Definition. A set of rules that define the valid structure, elements, and attributes for an XML document. DitaCraft validates DITA files against DITA 1.3 DTDs.

Related concepts

Validation on page 53

Appendix
Q

Guide Validation

End-to-end validation of an entire DITA guide by running the DITA Open Toolkit against the root map. Unlike file-level validation, guide validation detects cross-file issues such as broken references, missing images, and unresolved keys that only surface during the full DITA-OT processing pipeline. Results are displayed in a WebView report panel.

Appendix
R

href

Hypertext reference. A DITA attribute that specifies the location of a referenced resource. Used in `<topicref>`, `<xref>`, `<link>`, and other elements. Supports Ctrl+Click navigation in DitaCraft.

Related concepts

Smart Navigation on page 49

Appendix
S

Internationalization (i18n)

The process of designing software so that it can be adapted to various languages without engineering changes. DitaCraft supports localized diagnostic messages in English and French, with the language automatically detected from your VS Code display language setting.

Appendix
T

Keyboard Shortcuts

Key combinations that provide quick access to DitaCraft commands and VS Code features. Primary shortcuts include **Ctrl+Shift+V** (validate), **Ctrl+Shift+H** (preview), and **Ctrl+Click** (navigate).

Related reference

Keyboard Shortcuts on page 129

Appendix
U

keydef

Key definition. A DITA map element that defines a key and its associated resource or value. Keys enable indirect addressing and centralized management of reusable content.

Related concepts

Key Resolution on page 61

Appendix
V

keyref

Key reference. A DITA attribute that references a key defined in a map. When processed, the keyref is resolved to the key's target value or resource. Supports Ctrl+Click navigation in DitaCraft.

Related concepts

Key Resolution on page 61

Smart Navigation on page 49

Appendix
W

Key Scope

A DITA 1.3 feature (via the `@keyscope` attribute) that enables different key definitions within different contexts of the same map. Key scopes allow the same key name to resolve to different targets depending on the context.

DitaCraft implements the full DITA 1.3 keyscope algorithm: scope names on map elements trigger PushDown inheritance so all keys in that branch gain scope-qualified aliases (e.g., `admin.audience`). Placing `@keyscope` on a non-map topicref creates an inline scope branch. Multiple scope names can be declared on a single element (space-separated) to generate aliases under each name. Keyref chains inside scoped branches are resolved with the correct scope prefix.

Appendix X

Key Space

The collection of all key definitions available in a DITA map and its submaps, built using a DITA 1.3 spec-compliant breadth-first traversal. DitaCraft builds the key space from the root map and makes it available for resolving `keyref` and `conkeyref` attributes.

The key space includes scope-qualified aliases generated from `@keyscope` attributes (e.g., `product-a.install-guide`) and supports multi-hop keyref chains. An explosion cap of 50,000 entries prevents runaway alias expansion in deeply nested maps. The key space is cached with a 5-minute TTL and automatically invalidated when map files change.

Related concepts

Key Resolution on page 61

Appendix
Y

Live Preview

A DitaCraft feature that renders DITA content as HTML5 in real-time. The preview updates automatically as you edit and supports bidirectional scroll synchronization. Shortcut: **Ctrl+Shift+H**.

Related concepts

Live Preview on page 57

Related reference

Preview Settings on page 127

Appendix Z

Profiling

A DITA mechanism for filtering and flagging content based on attribute values. Profiling attributes such as audience, platform, product, and otherprops allow conditional processing where content is included or excluded based on the target output. DitaCraft validates profiling values against subject scheme constraints when available.

Related concepts

Profiling and Subject Scheme Validation on page 75

DITAVAL Support on page 65

Appendix
AA

Publishing Profile

A saved DITA-OT publish configuration — transtype, output directory, DITAVAL filter, and extra arguments — that can be selected instead of re-entering the same options on every publish. Managed via **DITA: Manage Publishing Profiles** and stored in the `ditacraft.publishingProfiles` workspace setting.

Related concepts

Publishing Workflow Enhancements on page 107

Appendix
AB

Map Visualizer

A DitaCraft feature that displays your DITA map hierarchy as an interactive tree diagram. Click any node to navigate to the corresponding file. The visualizer updates in real-time as you modify the map.

Related concepts

Map Visualizer on page 59

Appendix
AC

Rate Limiting

A DitaCraft protection mechanism that limits validation requests to prevent performance issues during rapid editing. Maximum 10 requests per second per file. Excess requests during auto-validation are silently skipped.

Related concepts

Validation on page 53

Related reference

Validation Commands on page 27

Appendix
AD

Reference

A DITA topic type designed for reference information such as API documentation, command syntax, or configuration parameters. Uses the `<reference>` root element with `<refbody>`.

Related reference

File Creation Commands on page 31

Appendix
AE

RNG (RelaxNG)

A schema language for XML that provides an alternative to DTD for defining document structure. DitaCraft supports optional RNG validation via the `salve-annos` library. Enable it by setting `ditacraft.schemaFormat` to `"rng"` and providing a path to RNG schema files.

Appendix
AF

Root Map

The top-level DITA map that defines the scope for key resolution, cross-reference validation, and subject scheme discovery. DitaCraft can auto-discover root maps or you can set one explicitly via the `DITA: Set Root Map` command. The root map determines which keys are available and which profiling constraints apply.

Appendix
AG

Scroll Sync

Bidirectional scroll synchronization between the editor and preview panel. When enabled, scrolling in either pane automatically scrolls the other to the corresponding position. Controlled by `ditacraft.previewScrollSync`.

Related concepts

Live Preview on page 57

Related reference

Preview Settings on page 127

Appendix
AH

Subject Scheme

A type of DITA map that defines controlled vocabularies for profiling attributes. Subject scheme maps specify allowed values for attributes such as audience, platform, product, and otherprops. DitaCraft uses subject schemes to validate that profiling attribute values conform to the defined vocabulary.

Related concepts

Profiling and Subject Scheme Validation on page 75

Appendix
AI

Smart Navigation

A DitaCraft feature that enables Ctrl+Click navigation on DITA reference attributes including href, conref, keyref, and conkeyref. Clicking a reference opens the target file and positions the cursor at the referenced element.

Related concepts

Smart Navigation on page 49

Related reference

Navigation Commands on page 37

Appendix
AJ

Task

A DITA topic type designed for procedural information with step-by-step instructions. Tasks explain "how to" perform actions. Uses the `<task>` root element with `<taskbody>` containing `<steps>`.

Related reference

File Creation Commands on page 31

Appendix
AK

Topic

The basic unit of content in DITA. A topic is a standalone unit of information that addresses a single subject. DITA provides specialized topic types: concept, task, reference, and generic topic. File extension: `.dita`.

Related concepts

Introduction to DitaCraft on page 15

Related reference

File Creation Commands on page 31

Appendix
AL

topicref

A DITA map element that references a topic to include in the publication. The `<topicref>` element uses the `href` attribute to specify the topic file and can be nested to create hierarchy.

Related concepts

Map Visualizer on page 59

Related reference

File Creation Commands on page 31

Appendix
AM

TypesXML

A pure TypeScript XML parser used by DitaCraft for DTD validation. TypesXML provides 100% W3C conformance and full DTD support without requiring external dependencies. The default and recommended validation engine.

Related concepts

Validation on page 53

Related reference

Validation Settings on page 119

Appendix
AN

Validation

The process of checking DITA content for XML well-formedness and DTD conformance. DitaCraft provides real-time validation with detailed error messages and multiple validation engine options.

Related concepts

Validation on page 53

Related reference

Validation Commands on page 27

Validation Settings on page 119

Appendix
AO

Validation Engine

A component that performs XML and DTD validation. DitaCraft supports three engines: TypesXML (default, full DTD support), built-in (lightweight, fast), and xmllint (external, industry-standard). Configured via `ditacraft.validationEngine`.

Related concepts

Validation on page 53

Related reference

Validation Settings on page 119

Appendix
AP

Watch Mode

A DitaCraft mode, started with **DITA: Start Watch Mode**, that automatically re-runs a full DITA-OT publish whenever a watched DITA file changes. Not incremental — DITA-OT has no first-class incremental build mode, so every change triggers a complete republish.

Related concepts

Publishing Workflow Enhancements on page 107

Appendix
AQ

xmllint

A command-line XML validation tool from the libxml2 library. DitaCraft can use xmllint as an external validation engine. Provides excellent performance and accuracy but requires separate installation on your system.

Related reference

Validation Settings on page 119